Tutoriel Apache Hop

Construire un pipeline ETL complet et l'orchestrer avec un workflow

Data Engineering et préparation de données – Master 2 Data Science

Thomas

15 octobre 2026

Résumé

Ce tutoriel guide pas à pas la construction d'un pipeline ETL complet avec Apache Hop, à partir de trois fichiers CSV de commerce en ligne : nettoyage des données, jointures, calcul du chiffre d'affaires, agrégations et export des résultats, puis orchestration de l'ensemble par un workflow. Il présente les concepts essentiels d'Apache Hop (projet, pipeline, transformation, hop, workflow, action), explique le rôle de chaque transformation utilisée et illustre chaque étape par des captures d'écran. Une dernière partie, en autonomie, propose de prolonger le travail avec la segmentation des clients « premium » et un workflow étendu.

Le résultat à atteindre. La figure 1 montre le pipeline que nous allons construire. Trois branches de lecture (commandes, clients, produits) sont nettoyées, puis jointes ; le chiffre d'affaires est calculé, agrégé de quatre façons différentes (par date, ville, client et produit) et exporté dans des fichiers. Chaque brique de ce schéma est expliquée dans les étapes suivantes.

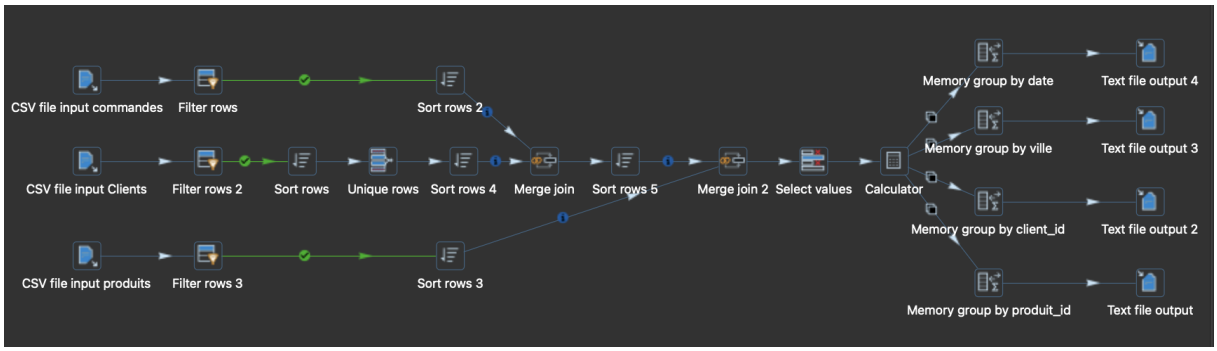


FIGURE 1 – Vue d'ensemble : le pipeline ETL complet construit dans ce tutoriel.

Table des matières

1 Introduction **4**
1.1 Contexte . . . 4
1.2 Structure des données . . . 4
1.3 Le traitement visé . . . 4

2 Les concepts clés d'Apache Hop **4**
2.1 Pipeline et workflow : la différence essentielle . . . 5
2.2 Le bouton « Get fields » . . . 5

3 Étape 1 – Prise en main et import des données **6**
3.1 L'interface de Hop GUI . . . 6
3.2 Création du projet . . . 6
3.3 Création du premier pipeline . . . 9
3.4 La transformation *CSV file input* . . . 9
3.5 Lancement rapide et inspection des données . . . 11
3.6 Lecture des erreurs . . . 11

4 Étape 2 – Nettoyage des données **13**
4.1 La transformation *Filter rows* . . . 13
4.1.1 Nettoyage des commandes . . . 14
4.1.2 Nettoyage des clients et des produits . . . 15
4.2 La transformation *Sort rows* . . . 16
4.3 La transformation *Unique rows* . . . 17

5 Étape 3 – Jointures et calcul du chiffre d'affaires **20**
5.1 La transformation *Merge join* . . . 20
5.2 Jointure commandes – clients . . . 20
5.3 Jointure avec les produits . . . 21
5.4 La transformation *Select values* . . . 22
5.5 La transformation *Calculator* . . . 24

6 Étape 4 – Agrégations et export des résultats **25**
6.1 La transformation *Memory group by* . . . 25
6.2 Chiffre d'affaires par dimension . . . 27
6.3 La transformation *Text file output* . . . 29
6.4 Pipeline ETL complet . . . 29

7 Étape 5 – Orchestration avec un workflow **30**
7.1 Création du workflow . . . 30
7.2 L'action *Pipeline* . . . 31
7.3 Les hops de workflow . . . 33
7.4 Exécution du workflow . . . 34

8 Étape 6 – En autonomie : segmentation premium **34**
8.1 Énoncé . . . 34
8.2 Pistes . . . 35
8.3 Solution proposée . . . 35
8.4 Pipelines complémentaires et workflow étendu . . . 35

9 Questions de synthèse **36**

10 Bilan **37**

1 Introduction

1.1 Contexte

Vous travaillez comme *data engineer* dans une entreprise de commerce en ligne. Trois fichiers CSV indépendants sont produits quotidiennement :

— `clients.csv` : les clients ;

— `produits.csv` : le catalogue de produits ;

— `commandes.csv` : les commandes passées.

Le service décisionnel souhaite obtenir des indicateurs de ventes : chiffre d'affaires par client, par produit et par ville. Or les données brutes contiennent volontairement plusieurs problèmes : valeurs manquantes, doublons, identifiants inexistants, quantités invalides, dates incorrectes, données incohérentes. L'objectif est de construire, avec Apache Hop, un traitement qui transforme ces données brutes en données exploitables.

Prérequis. Apache Hop est installé (voir `guide_d_installation_apache_hop.md`) et les trois fichiers CSV sont placés dans le dossier `data/`.

1.2 Structure des données

Fichier	Champs	Clé
`clients.csv`	`client_id`, `nom`, `email`, `ville`, `date_inscription`	`client_id`
`produits.csv`	`produit_id`, `produit`, `categorie`, `prix`	`produit_id`
`commandes.csv`	`commande_id`, `client_id`, `produit_id`, `date`, `quantite`	`commande_id`

TABLE 1 – Description des trois fichiers sources.

1.3 Le traitement visé

Le chemin complet d'une donnée est le suivant :

$$\text{CSV brut} \rightarrow \text{Nettoyage} \rightarrow \text{Jointure} \rightarrow \text{Transformation} \rightarrow \text{Agrégation} \rightarrow \text{Résultat}$$

avec le calcul du chiffre d'affaires :

$$\text{CA} = \text{prix} \times \text{quantité}$$

2 Les concepts clés d'Apache Hop

Apache Hop (*Hop Orchestration Platform*) est une plateforme open source d'intégration de données. Elle permet de concevoir visuellement, par glisser-déposer, des traitements de données (ETL/ELT) sans écrire de code.

Vocabulaire fondamental

Projet	Espace de travail qui regroupe les fichiers, les pipelines, les workflows et la configuration (variables, environnements).
Pipeline (`.hpl`)	Graphe de *transformations* qui traite des données ligne par ligne :

	lecture, nettoyage, jointure, calcul, agrégation, écriture.
Transformation	Brique élémentaire d'un pipeline (par exemple *CSV file input*, *Filter rows*, *Merge join*). Elle reçoit un flux de lignes et en produit un autre.
Hop	Flèche qui relie deux transformations (ou deux actions) et indique le sens de circulation des données (ou de l'exécution).
Workflow (`.hwf`)	Graphe d'*actions* qui orchestre l'exécution : ordre, conditions de succès ou d'échec.
Action	Brique élémentaire d'un workflow (par exemple *Start*, *Pipeline*, *Success*).

2.1 Pipeline et workflow : la différence essentielle

	Pipeline	**Workflow**
Extension	`.hpl`	`.hwf`
Éléments	Transformations	Actions
Ce qui circule	Des lignes de données (flux)	Un ordre d'exécution (succès ou échec)
Exécution	Toutes les transformations tournent en parallèle	Les actions s'exécutent séquentiellement
Rôle	**Transformer** les données	**Orchestrer** les traitements

TABLE 2 – Pipeline et workflow.

Attention

Dans un pipeline, toutes les transformations démarrent simultanément et les lignes circulent en flux. Cela explique pourquoi certaines transformations (tri, jointure) ont des contraintes particulières, détaillées plus loin.

2.2 Le bouton « Get fields »

De nombreuses fenêtres de configuration (*CSV file input*, *Sort rows*, *Unique rows*, *Select values*, *Memory group by*, *Merge join*...) comportent un bouton **Get fields** (« récupérer les champs »), ou l'une de ses variantes : *Get key fields* (*Merge join*), *Get lookup fields* (agrégats de *Memory group by*), *Get fields to remove* (*Select values*). Ce bouton remplit automatiquement le tableau de la fenêtre avec la liste des champs disponibles, au lieu de devoir les saisir un à un, ce qui évite les fautes de frappe.

Son comportement dépend de la transformation :

— dans une **transformation d'entrée** comme *CSV file input*, il lit les premières lignes du fichier et propose, pour chaque colonne, un nom et un **type deviné** (entier, texte, date...) ;

— dans une **transformation intermédiaire** (tri, déduplication, jointure, agrégation...), il récupère les champs du **flux entrant**, c'est-à-dire ceux produits par la transformation précédente.

Attention

— Pour une transformation intermédiaire, il faut d'abord créer le hop qui la relie à la transformation précédente : sans flux entrant, la liste de champs reste vide.
— Pour *CSV file input*, le séparateur, l'encodage et la présence d'un en-tête doivent être renseignés *avant* de cliquer, sinon les champs sont mal détectés.
— Les types devinés sont une proposition, pas une garantie : il faut les vérifier (nous verrons un exemple avec les dates).
— Le bouton ajoute tous les champs : on supprime ensuite les lignes inutiles. Si le fichier ou une transformation en amont change, il faut cliquer à nouveau.

3 Étape 1 – Prise en main et import des données

3.1 L'interface de Hop GUI

Hop GUI est l'environnement graphique de conception. La barre latérale gauche donne accès à l'explorateur de fichiers du projet, aux perspectives (métadonnées, exécution, etc.) ; la zone centrale est le canevas sur lequel on dessine pipelines et workflows ; un panneau en bas de l'écran affiche les logs et la prévisualisation des données. Au premier lancement, le projet actif est le projet `default`.

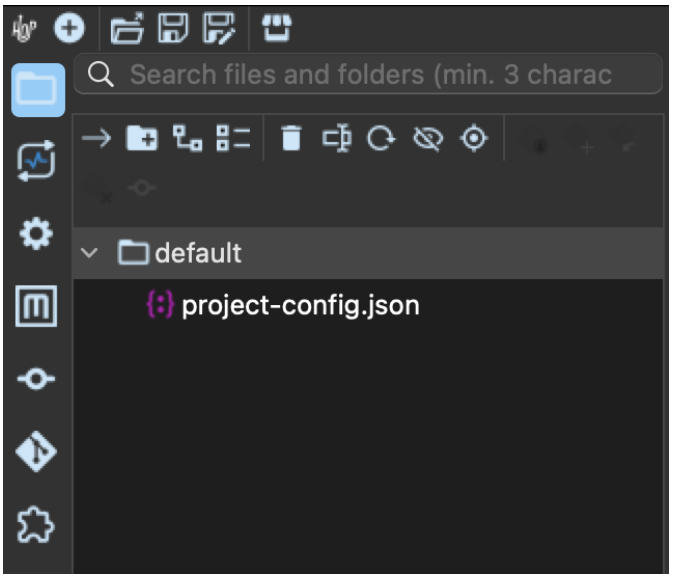


FIGURE 2 – Explorateur de fichiers de Hop GUI (projet `default`).

3.2 Création du projet

Un projet Apache Hop regroupe l'ensemble des fichiers d'un traitement. Nous créons un projet nommé `ecommerce_td`, dont le dossier racine est le dépôt `apache-hop-class`, organisé de la manière suivante :

```
apache-hop-class/
|-- data/
|   |-- ca_client.txt
|   |-- ca_date.txt
|   |-- ca_produit.txt
|   |-- ca_ville.txt
|   |-- clients.csv
|   |-- commandes.csv
```

```
|   |-- pipeline_test.hpl
|   |-- produits.csv
|   |-- workflow_test.hwf
|-- hop/
|-- guide_d_installation_apache_hop.md
```

Le dossier `data/` regroupe à la fois les sources (`clients.csv`, `produits.csv`, `commandes.csv`), les fichiers de résultats (`ca_*.txt`) et les fichiers Hop (`pipeline_test.hpl`, `workflow_test.hwf`). Au départ, seuls les trois CSV sont présents : les autres fichiers sont créés au fil du tutoriel. Le dossier `hop/` contient l'installation d'Apache Hop.

1. Cliquez sur « default » (en bas de la fenêtre) afin de créer un nouveau projet.



FIGURE 3 – Création du projet : clic sur le nom du projet actif (« default »).

2. Dans le menu qui s'ouvre, choisissez « Add project » puis « In a folder ».

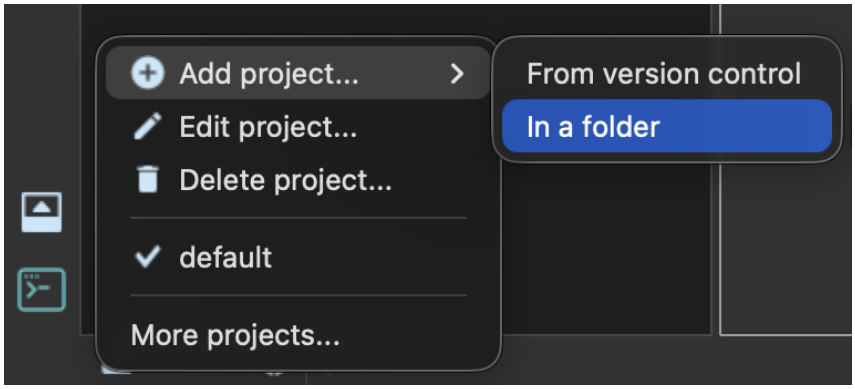


FIGURE 4 – Création du projet : menu « Add project » → « In a folder ».

3. Donnez le nom `ecommerce_td` au projet et choisissez son dossier de travail : ici, directement le dépôt `apache-hop-class`.

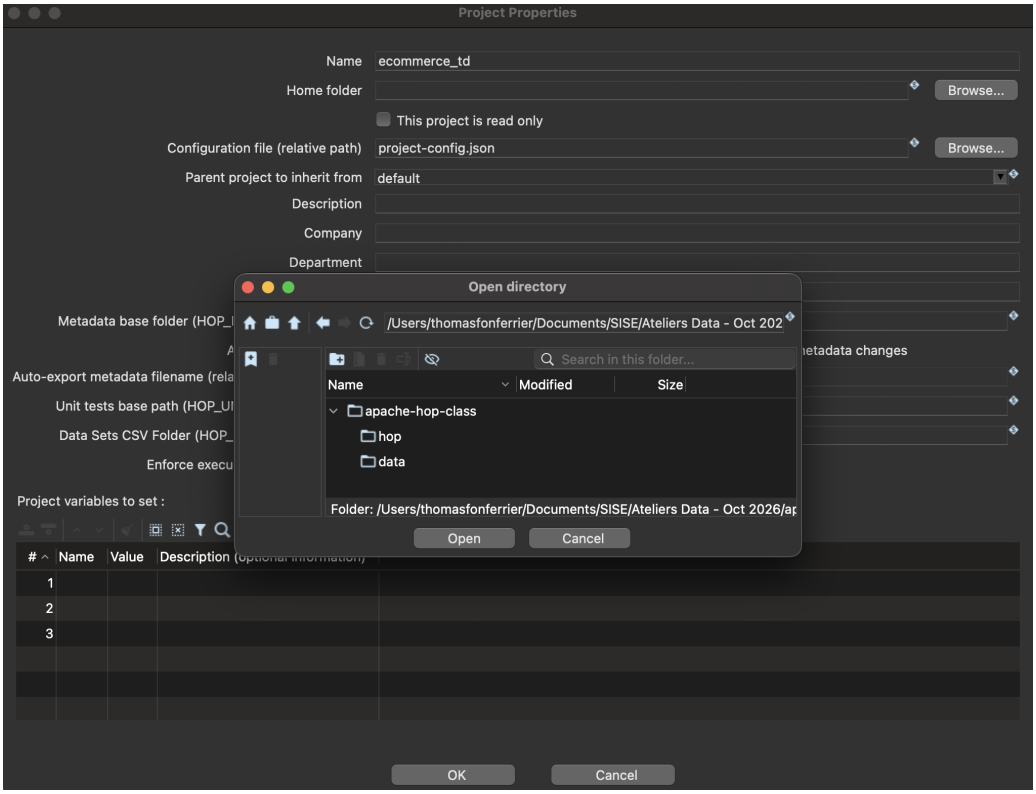


FIGURE 5 – Création du projet : propriétés et choix du dossier de travail.

4. Hop propose de créer un environnement de cycle de vie (*lifecycle environment*). Cliquez sur « No » : cette fonctionnalité n'est pas nécessaire pour ce cours.

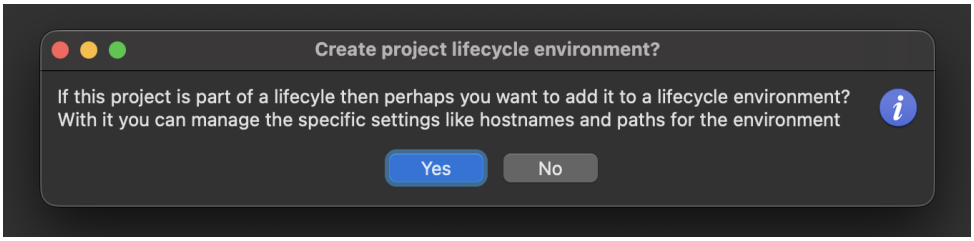


FIGURE 6 – Création du projet : proposition d'un environnement de cycle de vie.

Astuce

Tout est regroupé dans `data/`, ce qui rend l'arborescence compacte et garde les fichiers produits à côté de leurs sources. Dans tous les cas, il est recommandé d'exprimer les chemins de façon relative à la racine du projet, par exemple `${PROJECT_HOME}/data/commandes.csv`, afin que le projet reste déplaçable d'une machine à l'autre.

Question 1 – Quel fichier est la table centrale de l'analyse des ventes ?

La table centrale est `commandes.csv`. Chaque ligne représente un événement de vente (un client, un produit, une date, une quantité) et contient les clés étrangères `client_id` et `produit_id` qui permettent de rejoindre les deux autres tables. Les clients et les produits sont des tables de référence (dimensions) qui enrichissent les commandes (table de faits) : c'est à ce grain, une ligne par commande, que se calcule le chiffre d'affaires.

3.3 Création du premier pipeline

Cliquez sur « + » puis sur « Pipeline ».

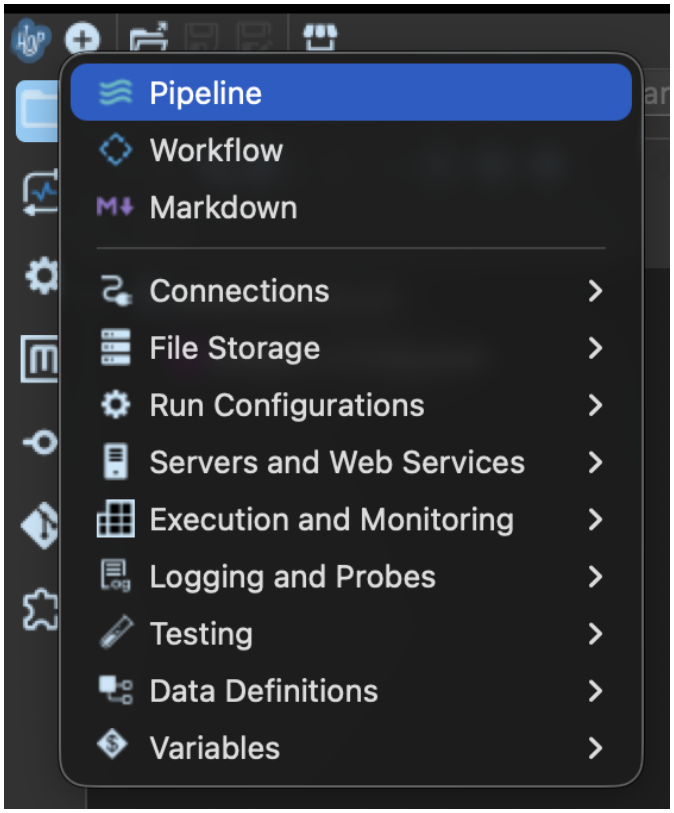


FIGURE 7 – Création d'un nouveau pipeline.

3.4 La transformation *CSV file input*

CSV file input

Transformation d'entrée qui lit un fichier CSV et produit un flux de lignes. Il faut préciser le fichier, le séparateur (virgule ou point-virgule), la présence d'un en-tête et l'encodage, puis cliquer sur *Get fields* pour détecter les champs et leurs types.

Cliquez n'importe où sur le canevas pour ouvrir la liste des transformations.

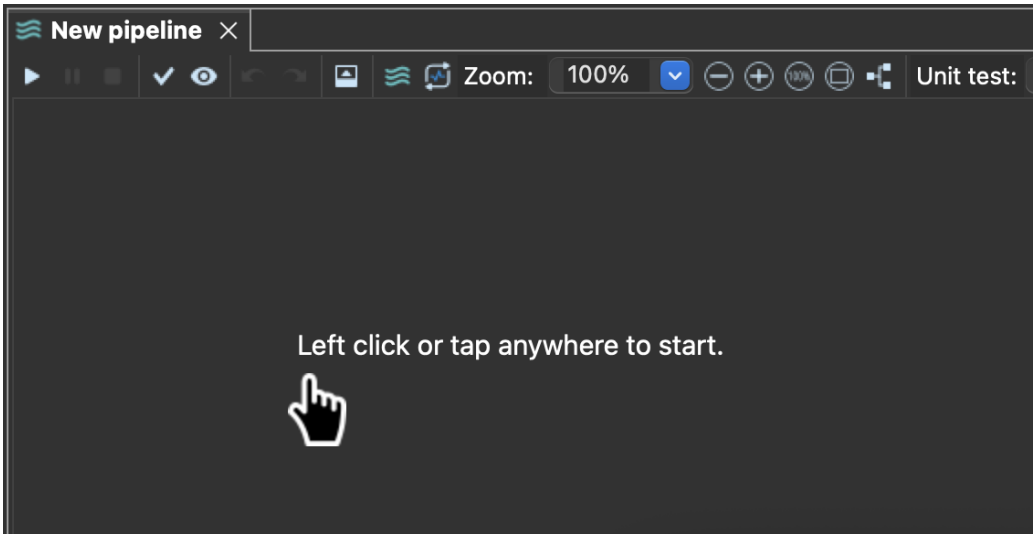


FIGURE 8 – Canevas vide d'un nouveau pipeline.

Saisissez « csv file input » dans la barre de recherche, puis sélectionnez la transformation.

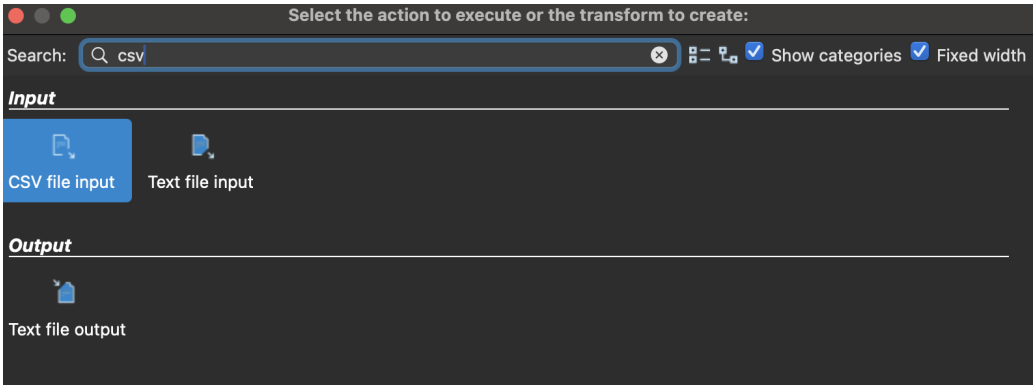


FIGURE 9 – Recherche et ajout de la transformation *CSV file input*.

Lorsqu'on clique sur le nom d'une transformation, on accède à sa configuration. Nous renseignons ici le fichier `commandes.csv`, le séparateur (virgule), la présence d'un en-tête, puis nous cliquons sur *Get fields* pour détecter les cinq champs : `commande_id`, `client_id`, `produit_id`, `date` et `quantite`.

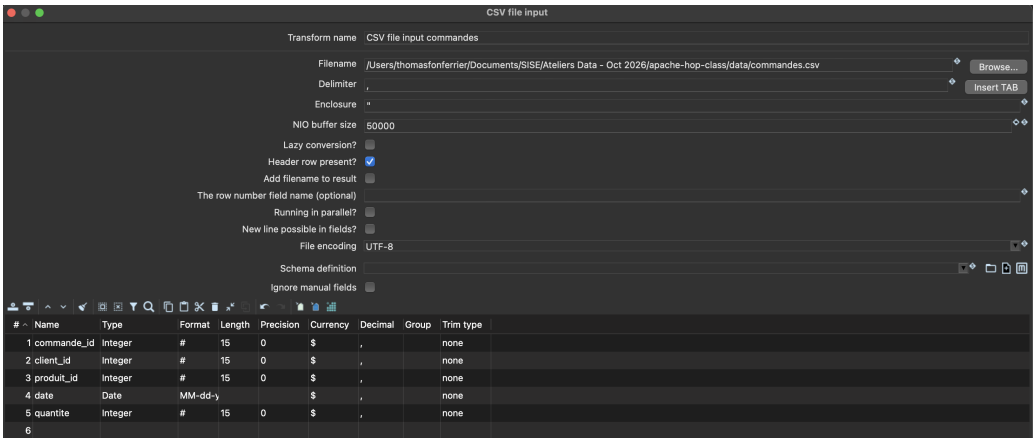


FIGURE 10 – Configuration de *CSV file input* pour `commandes.csv` : champs détectés.

Attention

Précisez bien l'encodage **UTF-8** afin d'éviter de futures erreurs de lecture. Pensez aussi à vérifier les types détectés : une date mal formatée ou un entier contenant du texte génère des erreurs.

3.5 Lancement rapide et inspection des données

Le lancement (*Run*, bouton en forme de triangle dans la barre d'outils) exécute le pipeline avec une *configuration d'exécution* (par défaut le moteur local de Hop). L'option de prévisualisation (*Preview*, icône en forme d'œil) permet d'inspecter les données produites par une transformation.

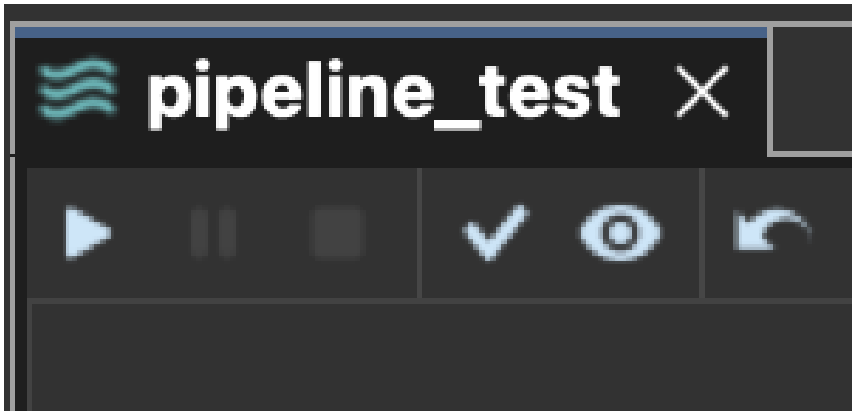


FIGURE 11 – Barre d'outils du pipeline : bouton de lancement (triangle) et de prévisualisation (œil).

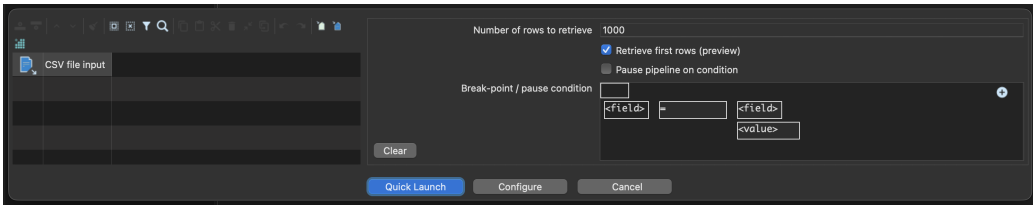


FIGURE 12 – Fenêtre de lancement rapide (*Quick Launch*) : nombre de lignes à prévisualiser.

Question 2

Combien de lignes contient le fichier `commandes.csv` ?

Question 3

Quels problèmes de qualité des données observez-vous déjà ?

3.6 Lecture des erreurs

Après ce premier lancement, vous devriez constater un message d'erreur. Lorsqu'un pipeline échoue, l'onglet de logs indique la transformation en cause et la nature de l'erreur (champ introuvable, conversion de type impossible, fichier absent…). Pour examiner de plus près les données produites par la transformation *CSV file input*, cliquez sur son icône (et non sur son nom), puis choisissez **View output**. D'après vous, qu'est-ce qui provoque l'erreur ?

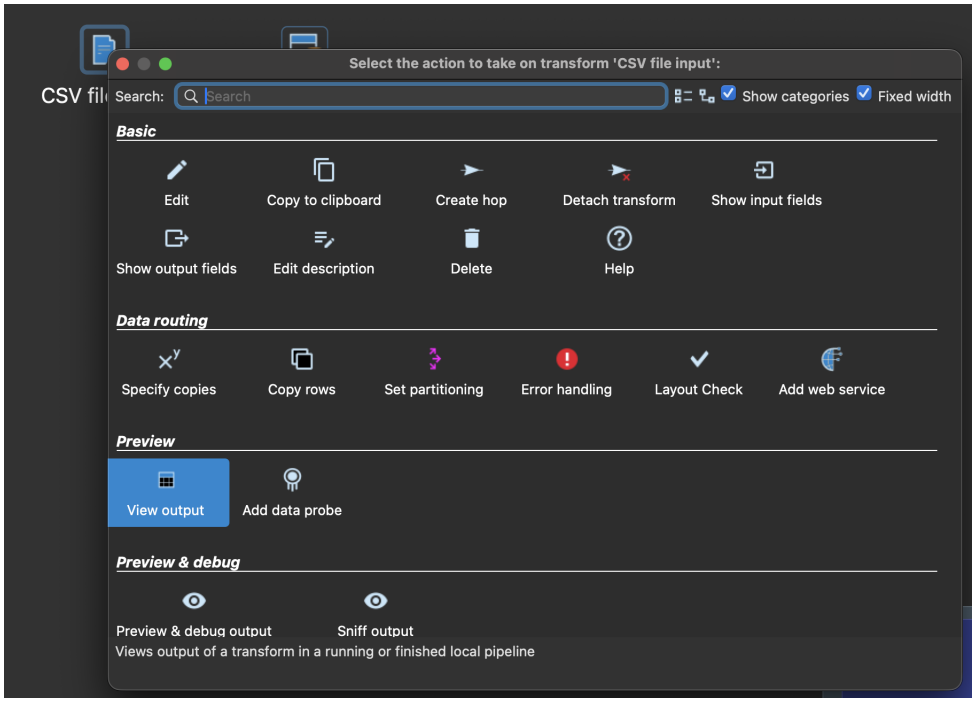


FIGURE 13 – Menu d'une transformation : action *View output*.

Il s'agit très probablement de dates aberrantes qui font échouer l'algorithme de conversion du logiciel. Pour résoudre ce problème, revenez dans la configuration de *CSV file input* et changez le type du champ `date` en **String** : Hop ne tentera plus de convertir ces valeurs en dates. La validité des dates sera vérifiée plus tard, lors du nettoyage.

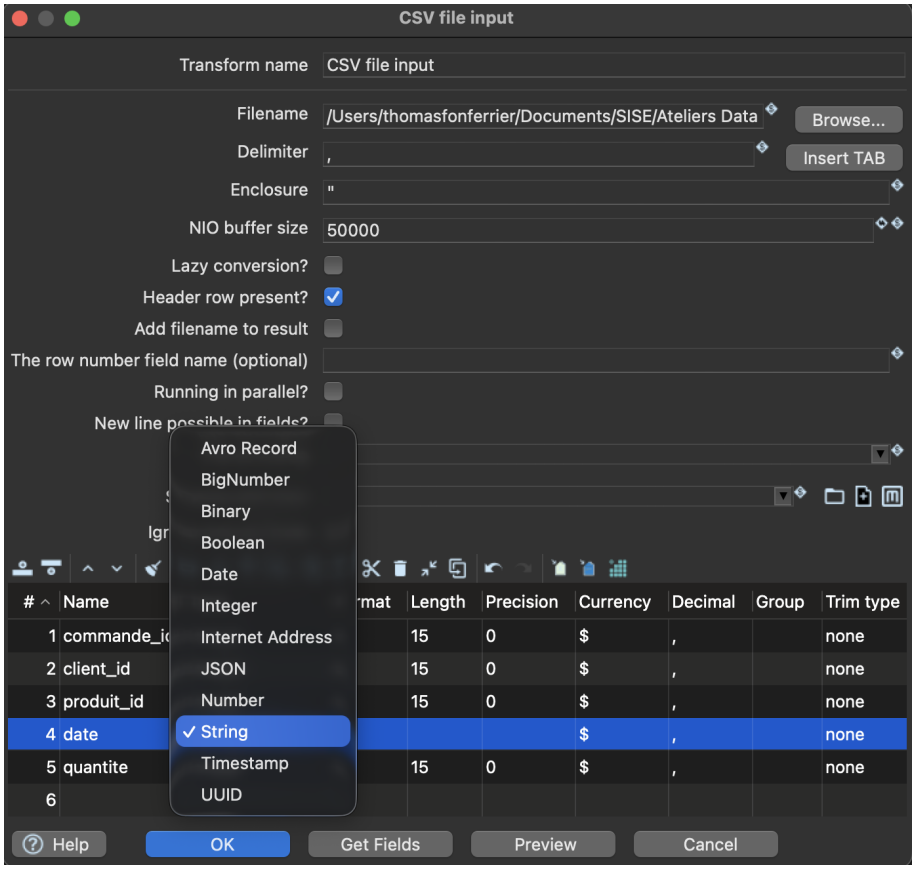


FIGURE 14 – Correction de l'erreur : le type du champ `date` est changé en *String*.

4 Étape 2 – Nettoyage des données

Le nettoyage est réalisé *avant* les jointures : des données invalides ou dupliquées fausseraient le résultat de la jointure et donc tous les calculs de chiffre d'affaires.

4.1 La transformation *Filter rows*

Filter rows

Cette transformation évalue une condition sur chaque ligne et oriente celle-ci selon le résultat : les lignes qui satisfont la condition (*true*) sont envoyées vers une transformation, celles qui ne la satisfont pas (*false*) vers une autre ou sont abandonnées. Elle joue le rôle d'un `WHERE` en SQL.

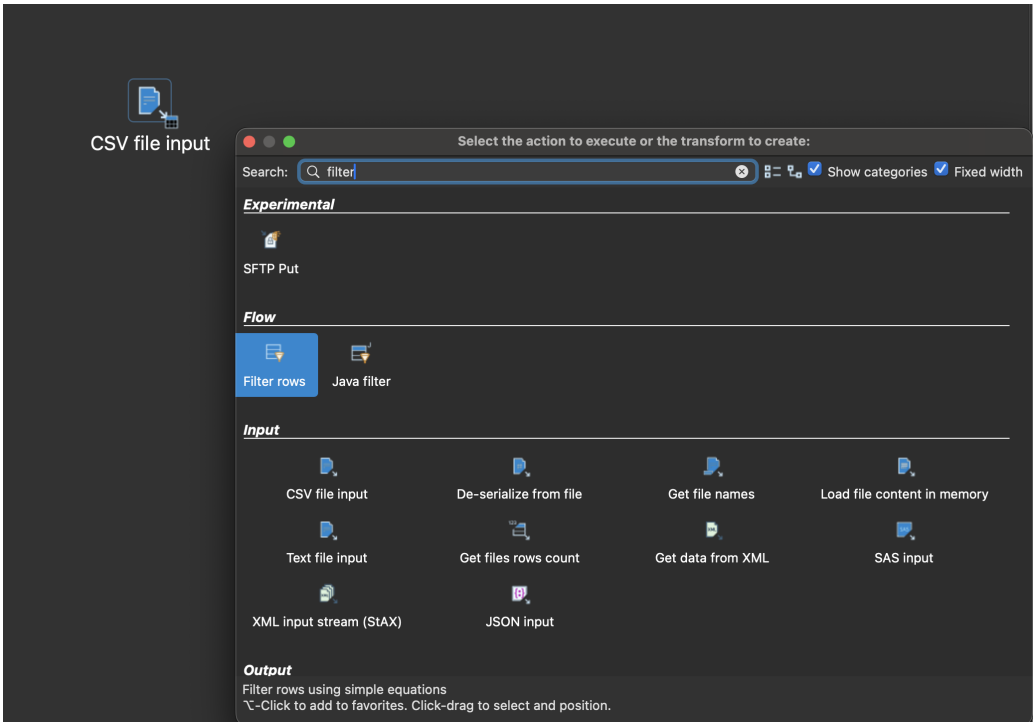


FIGURE 15 – Ajout de la transformation *Filter rows*.

Reliez *CSV file input* à *Filter rows* : cliquez sur l'icône de *CSV file input*, puis sur la flèche « hop » et faites-la glisser vers *Filter rows*.

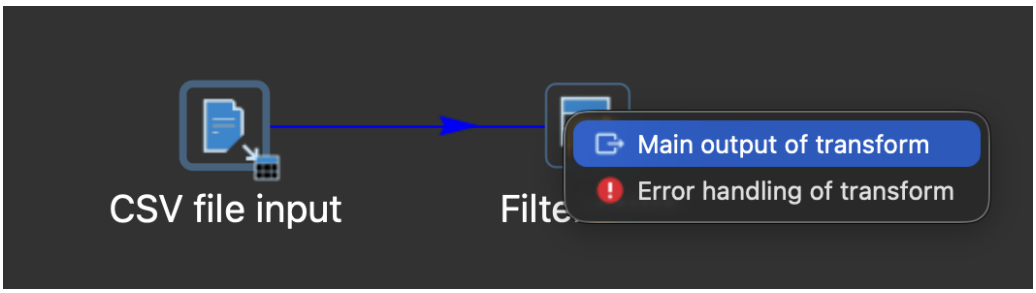


FIGURE 16 – Création d'un hop entre deux transformations.

4.1.1 Nettoyage des commandes

Une commande est considérée comme valide si elle respecte *simultanément* :

— une quantité strictement positive : `quantite` > 0 ;

— une quantité inférieure ou égale à 20 : `quantite` ≤ 20 ;

— une date valide : le champ `date`, lu comme texte, respecte le format `AAAA-MM-JJ`.

La condition du filtre combine ces critères par des opérateurs `AND` :

```
date REGEXP ^[0-9]{4}-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])$
AND quantite > 0
AND quantite <= 20
```

Cliquez sur le nom de *Filter rows* pour ouvrir sa configuration, puis implémentez le filtre sur la date avec l'opérateur `REGEXP` et l'expression régulière ci-dessus.

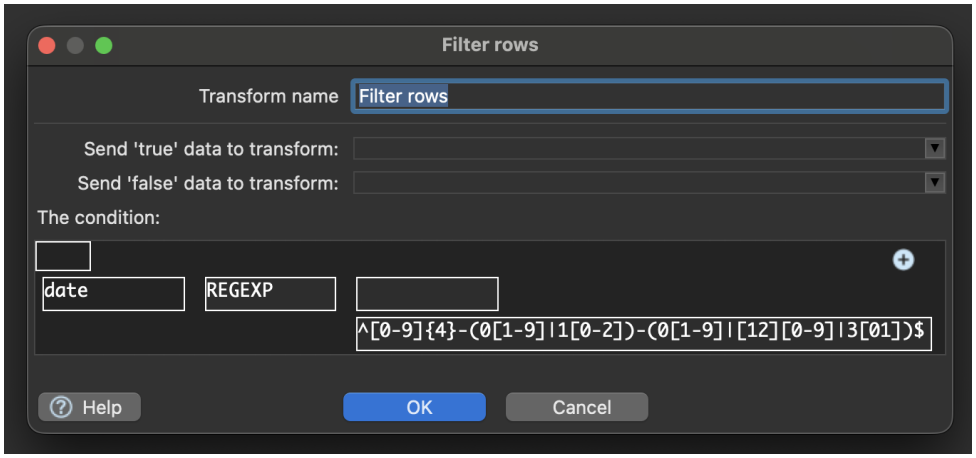


FIGURE 17 – Configuration du filtre sur la date des commandes (opérateur `REGEXP`).

Que fait cette expression régulière ?

— `^[0-9]{4}` : vérifie que le texte commence par quatre chiffres (l'année) ;

— `-(0[1-9]|1[0-2])` : vérifie que le mois est compris entre 01 et 12 (ce qui bloque, par exemple, un mois « 99 ») ;

— `-(0[1-9]|[12][0-9]|3[01])$` : vérifie que le jour est compris entre 01 et 31, puis que le texte se termine là.

Cette expression ne détecte pas les dates impossibles comme le 31 février, ce qui suffit pour cet exercice.

Un clic sur le bouton « + » permet d'ajouter les autres conditions (`quantite` > 0 et `quantite` ≤ 20), combinées par des `AND`.

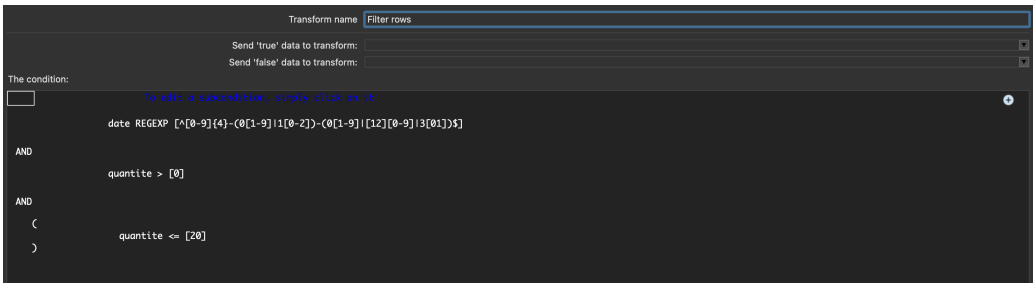


FIGURE 18 – Filtre complet sur les commandes : date, quantité strictement positive et inférieure ou égale à 20.

4.1.2 Nettoyage des clients et des produits

— **Clients** : traitement des villes et des emails manquants, puis suppression des doublons (voir *Unique rows*).

— **Produits** : suppression des produits dont le `prix` est manquant (simplification retenue pour cet exercice).

Attention

Supprimer une ligne n'est pas toujours la meilleure solution dans un projet réel : selon le contexte métier, on peut préférer imputer une valeur, marquer la ligne comme incomplète ou la mettre en quarantaine dans un flux d'erreurs.

Question 4 – Quelle stratégie pour les villes manquantes ?

Dans notre pipeline, les lignes clients dont la ville est absente sont éliminées par le filtre (*Filter rows 2*). Une alternative consiste à remplacer la valeur manquante par une valeur par défaut (par exemple « Inconnue ») pour ne pas perdre les commandes associées : c'est un arbitrage métier.

4.2 La transformation *Sort rows*

Sort rows

Trie le flux de lignes selon un ou plusieurs champs (ordre croissant ou décroissant). Elle est indispensable avant *Unique rows* et *Merge join*, qui supposent des données triées sur leurs clés.

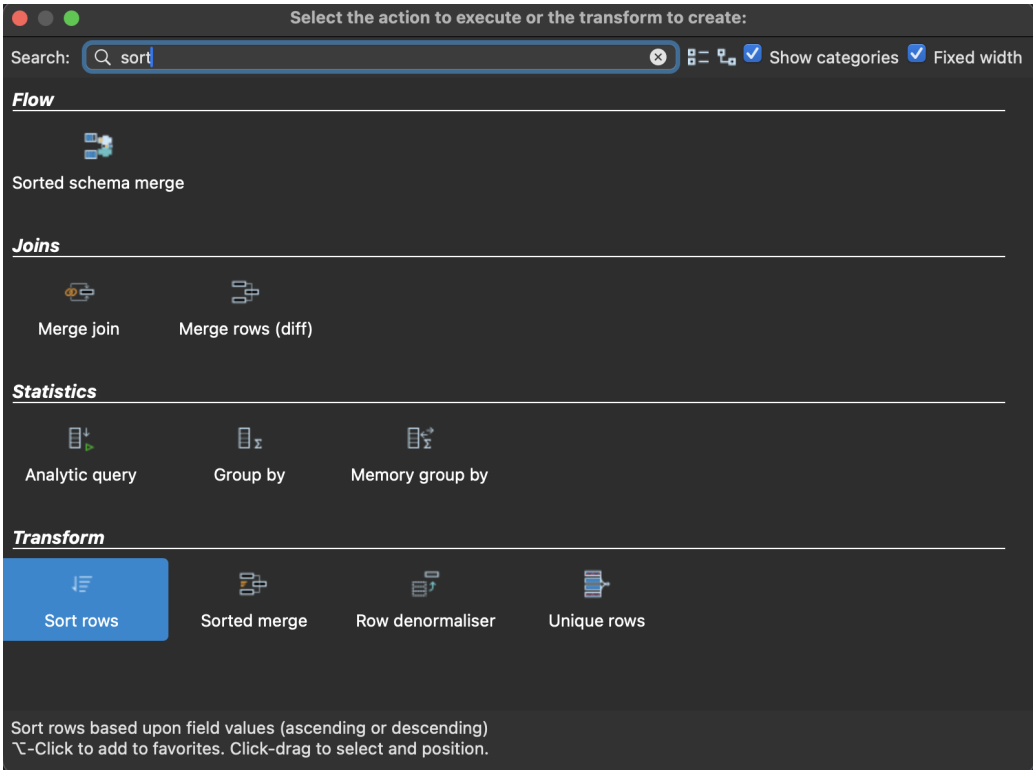


FIGURE 19 – Ajout de la transformation *Sort rows*.

Nous souhaitons supprimer les doublons de clients : nous triions donc les clients selon `client_id` (le bouton *Get fields* peut être utilisé pour remplir le tableau).

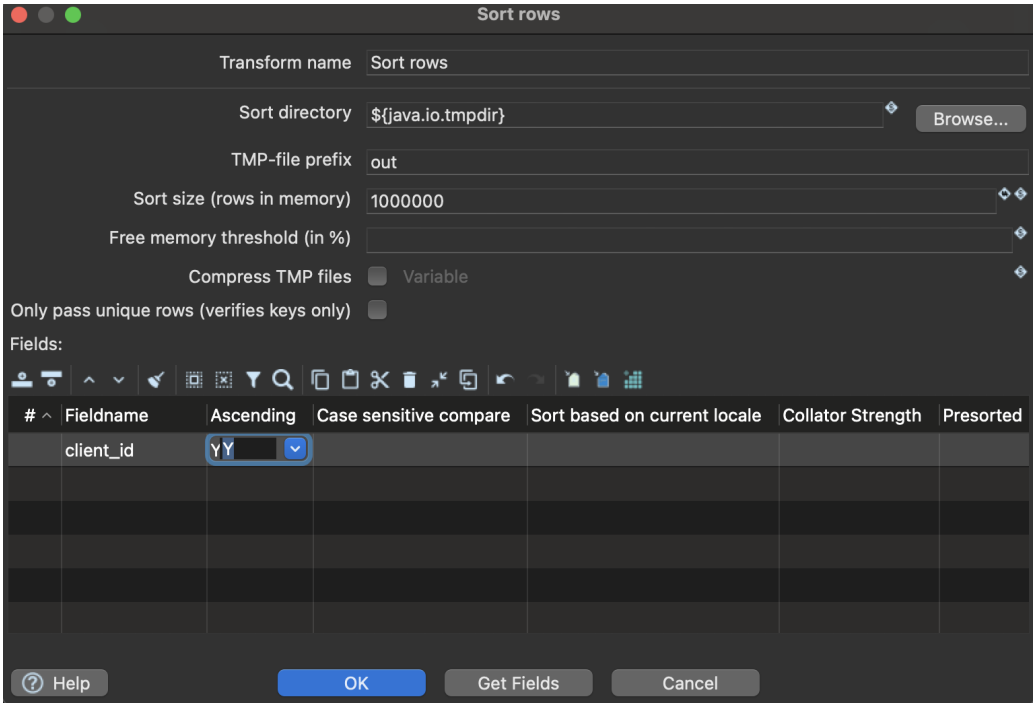


FIGURE 20 – Tri sur la clé `client_id`.

4.3 La transformation *Unique rows*

Unique rows

Supprime les lignes dupliquées. Elle compare chaque ligne à la précédente, c'est pourquoi le flux doit être trié. On précise les champs à comparer : s'il n'y en a aucun, la ligne entière est comparée.

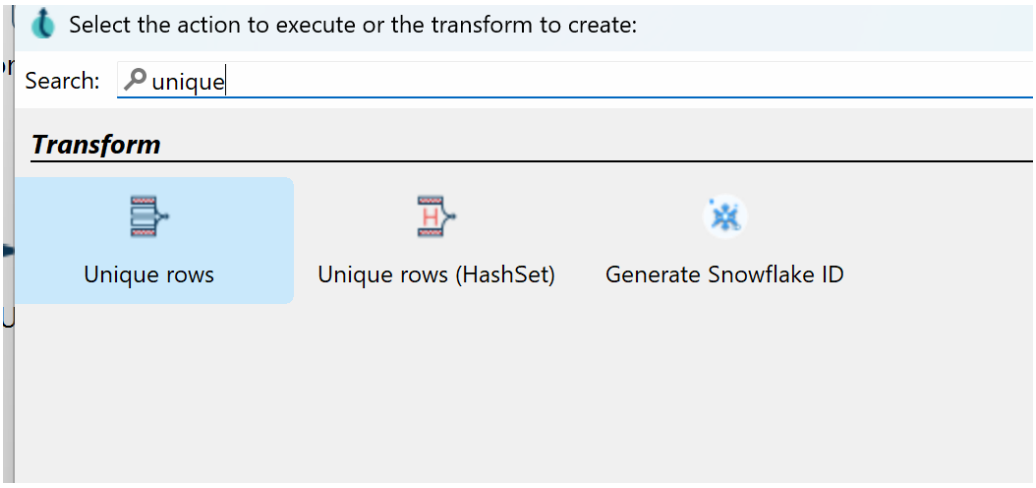


FIGURE 21 – Sélection de la transformation *Unique rows* dans le menu.

Ici, tous les champs du client sont comparés : deux lignes ne sont supprimées que si elles sont strictement identiques (l'option *Ignore case* permet d'ignorer la casse sur les champs texte).

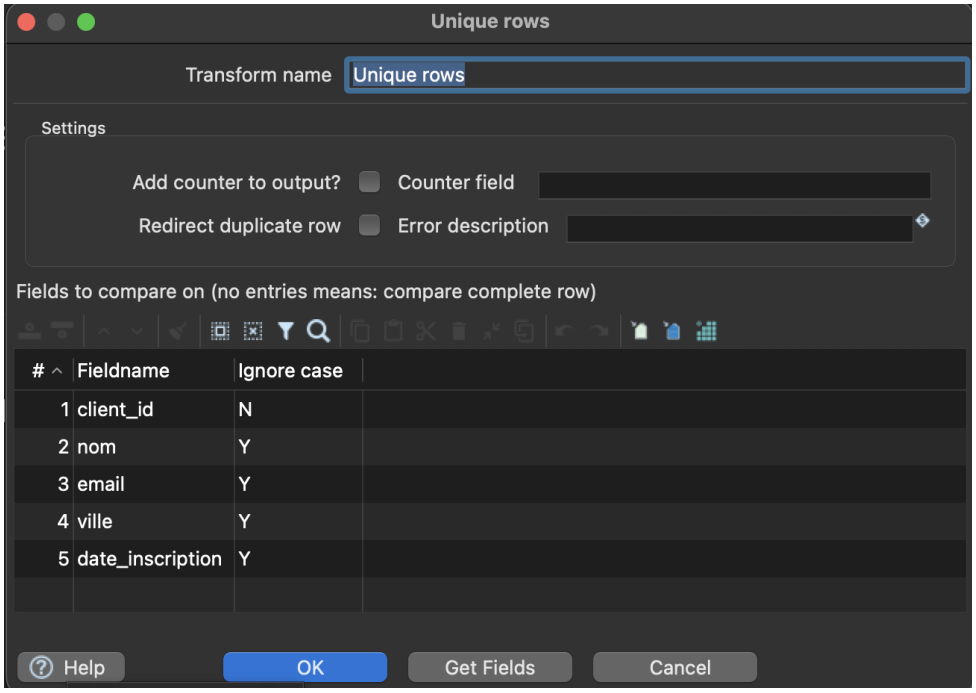


FIGURE 22 – Configuration de la déduplication des clients.

Question 5 – Pourquoi supprimer les doublons avant les jointures ?

Une jointure sur une clé qui n'est pas unique multiplie les lignes : si un client apparaît deux fois dans `clients.csv`, chacune de ses commandes apparaît deux fois après jointure. Le chiffre d'affaires est alors artificiellement doublé. La suppression des lignes strictement identiques réduit ce risque. Pour garantir une relation $n \to 1$, il faut également vérifier que chaque `client_id` restant est unique.

Unique rows (HashSet)

Supprime les doublons sans tri préalable en mémorisant les valeurs déjà rencontrées.

Sur la branche clients du même pipeline, remplacez le premier *Sort rows* → *Unique rows* par *Unique rows (HashSet)*. Comparez les mêmes champs, puis le nombre de lignes obtenu. Conservez le tri sur `client_id` avant *Merge join*.

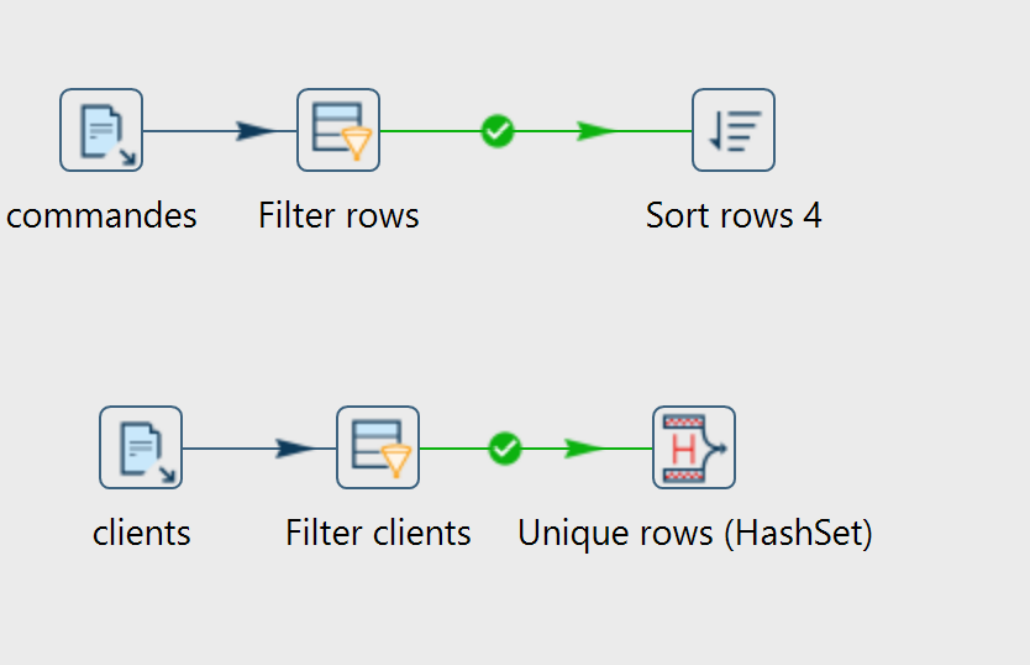


FIGURE 23 – Branche clients avec *Unique rows (HashSet)* : capture à ajouter.

Examine preview data

Rows of transform Unique rows (1000 rows)

#	client_id	nom	email	ville	date_inscripti
1	1	Alexandre Traore	patrick32@example.net	Rennes	2024-01-17
2	2	Aurore Maury-Briand	martinezeric@example.org	Paris	2024-06-07
3	3	Alexandrie Guilbert de la Auger	cvasseur@example.net	Lyon	2024-07-27
4	4	Édouard Rocher	rleblanc@example.org	Grenoble	2024-05-19
5	5	Caroline Morin	wpereira@example.com	Bordeaux	2025-07-19
6	6	Michel Regnier	hoareauedouard@example.net	Toulouse	2024-08-13
7	7	Maryse Verdier de la Fischer	lambertmargaud@example.org	Toulouse	2026-01-21
8	8	Gabriel Roussel	dlopes@example.net	Marseille	2026-03-29
9	9	Julien Monnier	margueriteda-costa@example.net	Grenoble	2026-07-19
10	10	Thibaut Bousquet	elisabethchauveau@example.org	Paris	2024-03-24
11	11	Laetitia Riou	mariannegermain@example.org	Rennes	2024-06-04
12	12	Hugues Maréchal	christophe39@example.net	Grenoble	2024-09-05
13	13	Nicole Weiss	qmeunier@example.org	Clermont-Ferrand	2024-06-23
14	14	Julie Letellier-Mary	zquillon@example.com	Nice	2024-05-24

Examine preview data

Rows of transform Unique rows (HashSet) (1000 rows)

#	client_id	nom	email	ville	date_ins
1	530	Guillaume Chauvin-Le Goff	catherine22@example.com	Toulouse	2025-07-18
2	495	Christiane Le Goff	wpelletier@example.org	Nantes	2024-03-25
3	395	Antoine Moulin-Fernandez	dferreira@example.net	Lille	2025-07-25
4	1491	Gilbert Devaux	clemencemunoz@example.net	Bordeaux	2026-03-06
5	484	Frédéric Monnier	noemile-gall@example.com	Nice	2026-02-06
6	1092	Eugène de la Muller	laurent77@example.net	Angers	2025-10-24
7	788	William Brun de Lemonnier	nathalie94@example.org	Paris	2025-08-16
8	614	Luc Maillet	denis82@example.com	Clermont-Ferrand	2024-09-05
9	1008	Geneviève Le Marion	spayet@example.net	Nantes	2025-11-14
10	1234	René Martins	lombardnath@example.com	Montpellier	2026-09-21
11	1456	Claudine Bousquet	gonzalezguy@example.net	Paris	2025-09-16
12	1358	Marc de la Brunel	wfabre@example.org	Angers	2026-08-05
13	1850	Denis du Prévost	vmoulin@example.org	Marseille	2026-01-09
14	1818	Pierre Lemaître	benoiteugene@example.com	Paris	2026-08-03
15	1453	Arthur Martin	josettemathieu@example.net	Lyon	2024-01-11
16	276	Joséphine Wagner	alexandregallet@example.com	Dijon	2026-02-09
17	125	Paul Da Silva	constancemaury@example.com	Angers	2026-05-07

FIGURE 24 – Configuration des champs comparés : capture à ajouter.

Résultat de la comparaison

Les deux méthodes donnent le même résultat si les champs et les règles de comparaison sont identiques. *HashSet* évite le premier tri ; le tri avant *Merge join* reste nécessaire.

5 Étape 3 – Jointures et calcul du chiffre d'affaires

5.1 La transformation *Merge join*

Merge join

Joint deux flux de lignes sur une ou plusieurs clés. Les deux entrées doivent obligatoirement être **triées sur les clés de jointure**, dans le même ordre. Le type de jointure est au choix : *INNER*, *LEFT OUTER*, *RIGHT OUTER* ou *FULL OUTER*. On renseigne la première transformation (*First transform*), la seconde (*Second transform*) et les champs clés de chaque côté.

Dans le pipeline complet (figure 1), des transformations *Sort rows* sont placées juste avant chaque jointure : elles trient les flux sur leurs clés de jointure.

5.2 Jointure commandes – clients

La clé de jointure est :

$$\text{commandes.client_id} = \text{clients.client_id}$$

On récupère au minimum le `nom` et la `ville` du client. Le flux des commandes (trié par `client_id`) est joint au flux des clients nettoyés et dédupliqués.

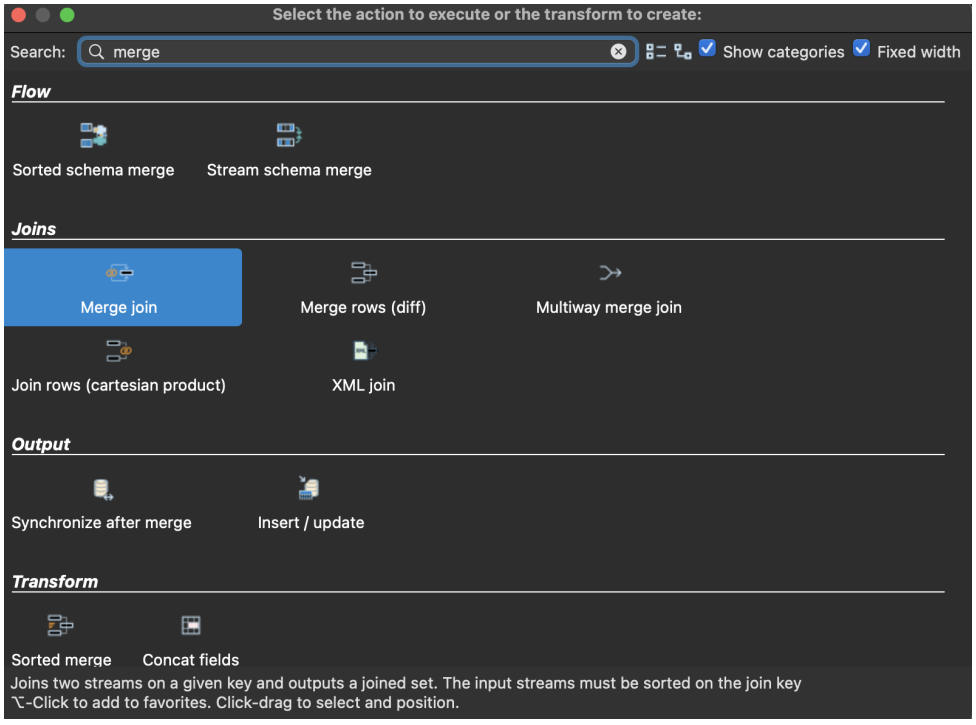


FIGURE 25 – Ajout de la première transformation *Merge join*.

On choisit ensuite, dans la configuration, les deux transformations à joindre et la clé de jointure de chaque côté.

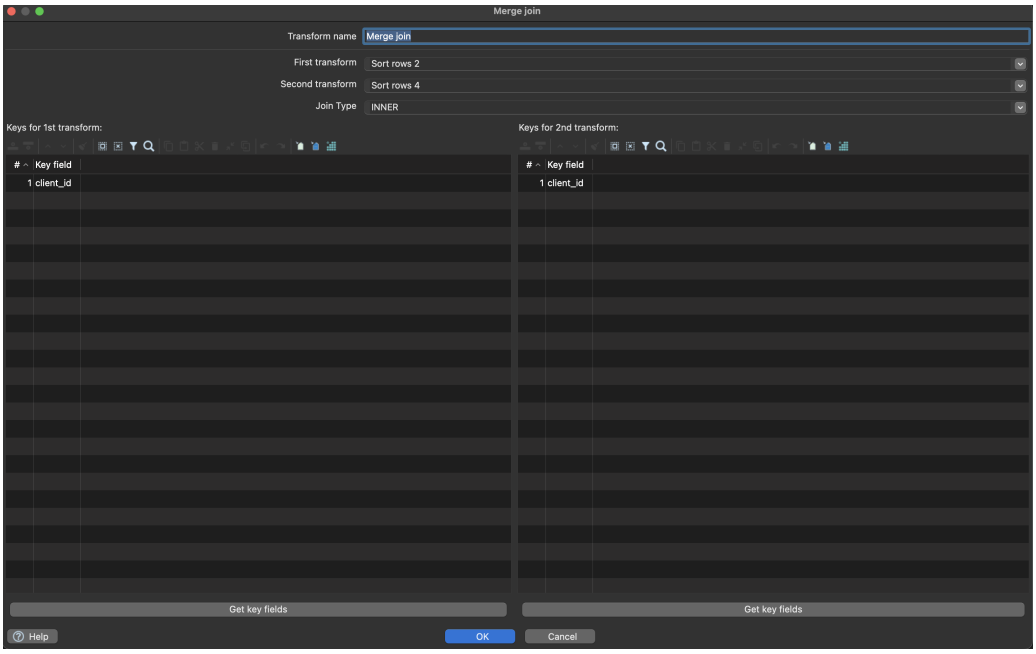


FIGURE 26 – Paramétrage de la jointure sur `client_id`.

Les deux flux contiennent un champ `client_id` : Hop renomme automatiquement celui du second flux en `client_id_1`. Nous le supprimerons plus loin avec *Select values*.

5.3 Jointure avec les produits

On re-trie le résultat de la première jointure sur `produit_id` (*Sort rows 5*), puis on le joint au flux des produits (trié lui aussi, *Sort rows 3*) :

$$\text{commandes.produit_id} = \text{produits.produit_id}$$

On récupère le `produit`, la `categorie` et le `prix`.

Stream lookup

Recherche, pour chaque ligne d'un flux principal, des valeurs dans un second flux chargé en mémoire à partir d'une clé commune.

Ici, utilisez `produit_id` pour récupérer `produit`, `categorie` et `prix` depuis les produits filtrés. Reliez à *Stream lookup* la sortie de la jointure commandes–clients et le flux des produits, puis comparez quelques commandes avec *Merge join 2*.

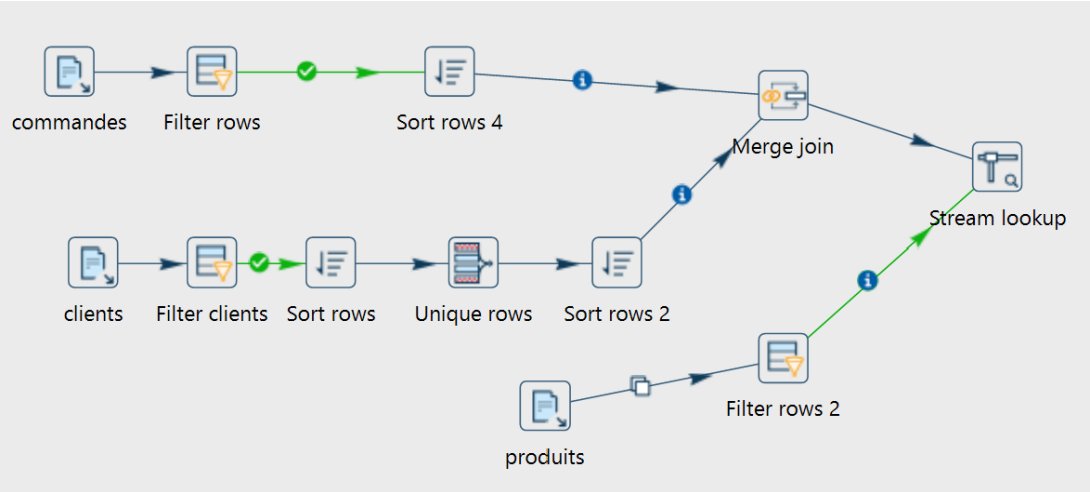


FIGURE 27 – Variante *Stream lookup*

Résultat de la comparaison

Pour un même `produit_id` présent dans les produits, les champs retrouvés sont les mêmes qu'avec *Merge join 2*. *Stream lookup* n'exige pas les deux tris sur cette clé, car il effectue sa recherche dans le flux produits chargé en mémoire.

5.4 La transformation *Select values*

Select values

Permet de sélectionner, réordonner, renommer ou supprimer des champs, et de modifier leurs métadonnées (type, format, longueur). Après les jointures, elle sert à ne conserver que les colonnes utiles : `commande_id`, `client_id`, `nom`, `ville`, `produit_id`, `produit`, `categorie`, `prix`, `quantite`, `date`. Ici, elle nous sert à supprimer `client_id_1` et, si ce champ existe encore après *Merge join 2*, `produit_id_1`.

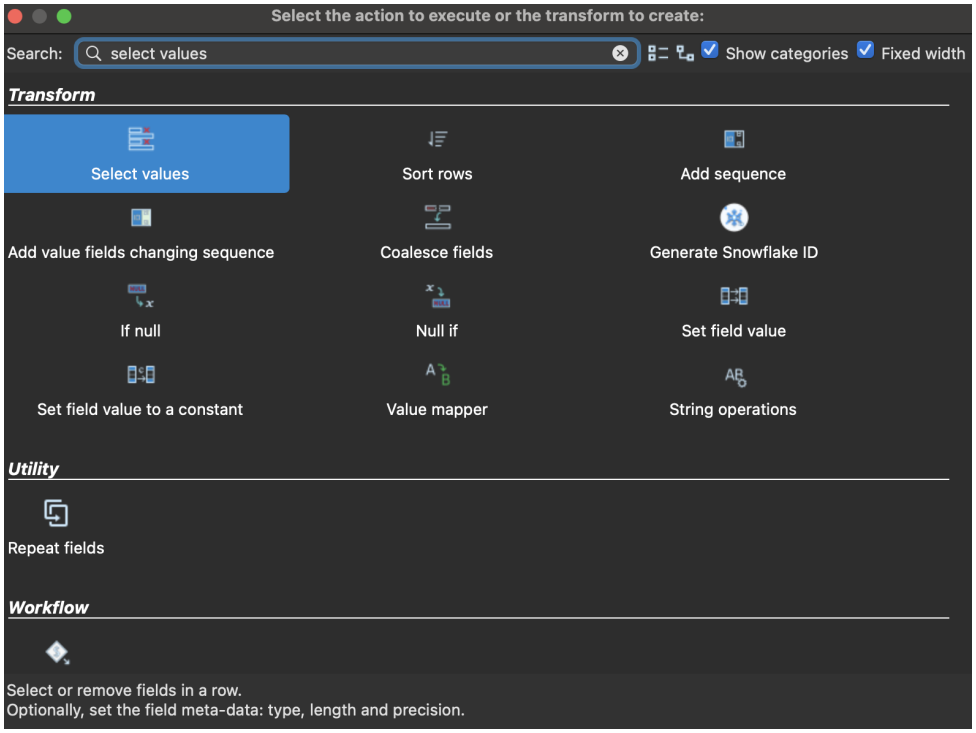


FIGURE 28 – Ajout de la transformation *Select values*.

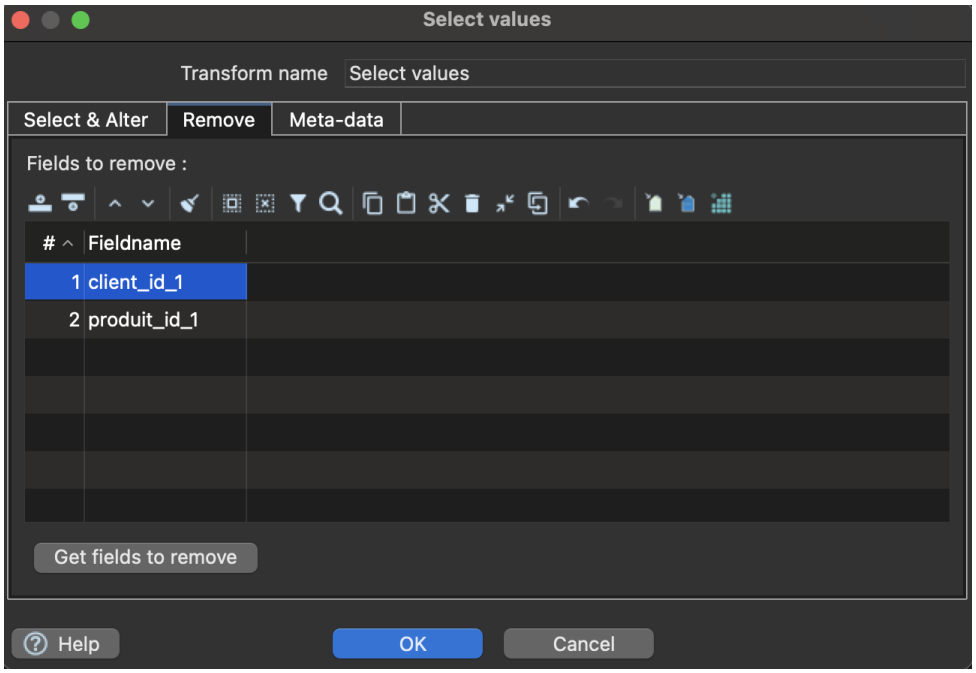


FIGURE 29 – Suppression des champs en double (onglet *Remove*).

5.5 La transformation *Calculator*

Calculator

Crée de nouveaux champs à partir d'opérations prédéfinies (addition, multiplication, concaténation, opérations sur les dates...). Chaque ligne de configuration définit : le nom du nouveau champ, le type de calcul, les champs opérandes A et B et le type du résultat.

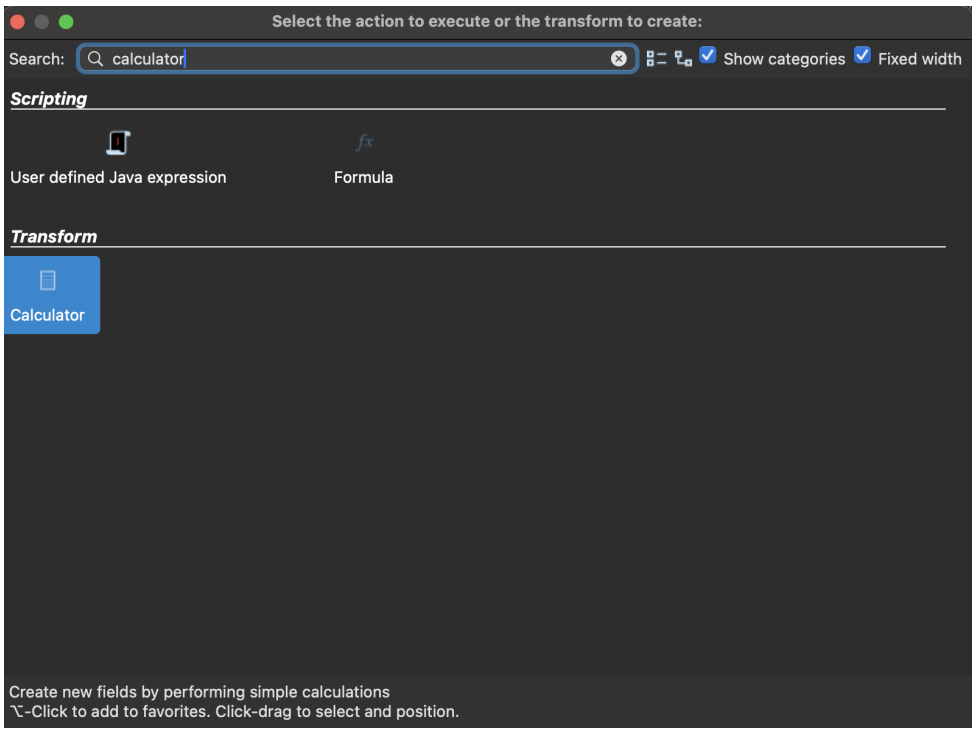


FIGURE 30 – Ajout de la transformation *Calculator*.

Pour le chiffre d'affaires, on crée le champ `CA` :

$$\texttt{CA} = \texttt{prix} \times \texttt{quantite}$$

avec le type de calcul *A * B*, A = `quantite`, B = `prix` et un type de résultat numérique (*Number*).

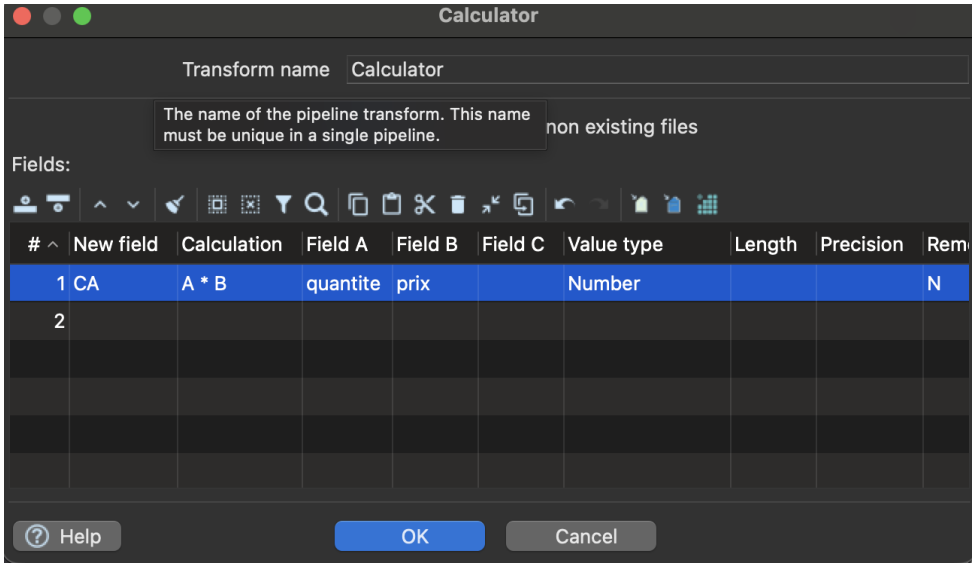


FIGURE 31 – Définition du champ `CA` = `quantite` × `prix`.

Astuce

Il est recommandé de prévisualiser le résultat et de vérifier à la main quelques lignes pour s'assurer que le calcul est correct.

6 Étape 4 – Agrégations et export des résultats

6.1 La transformation *Memory group by*

Memory group by

Réalise des agrégations (somme, moyenne, minimum, maximum, comptage…) par groupes, en mémoire : contrairement à *Group by*, elle **ne nécessite pas** de flux trié. On spécifie le ou les champs de regroupement (*Group fields*) et les agrégats (*Aggregates*) avec un nom de sortie, un champ source et un type d'agrégation. Elle correspond au `GROUP BY` de SQL.

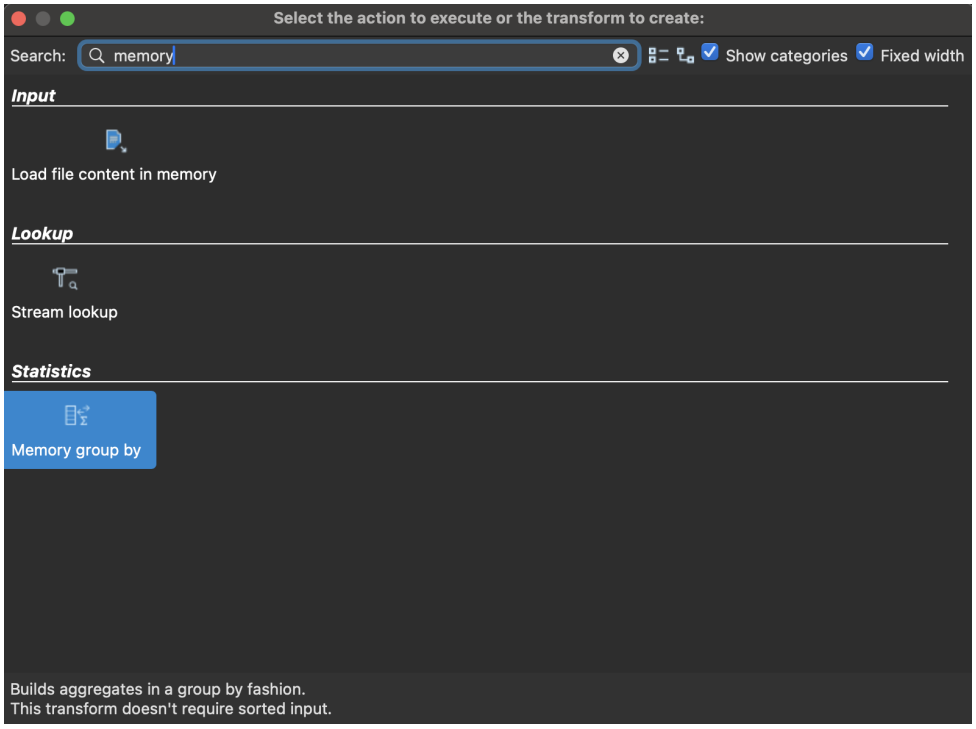


FIGURE 32 – Ajout de la transformation *Memory group by*.

Un premier *Memory group by*, sans champ de regroupement, calcule une agrégation sur l'ensemble des lignes : on crée un agrégat nommé `CA`, portant sur le champ `CA`, de type *Sum*.

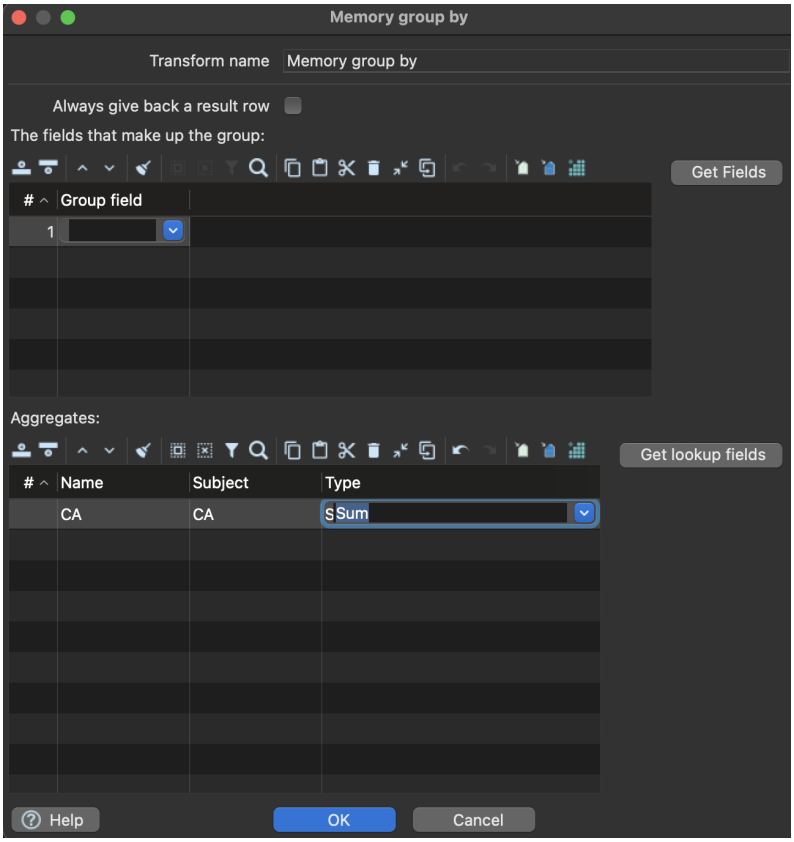


FIGURE 33 – Paramétrage d'une première agrégation : somme du champ `CA`.

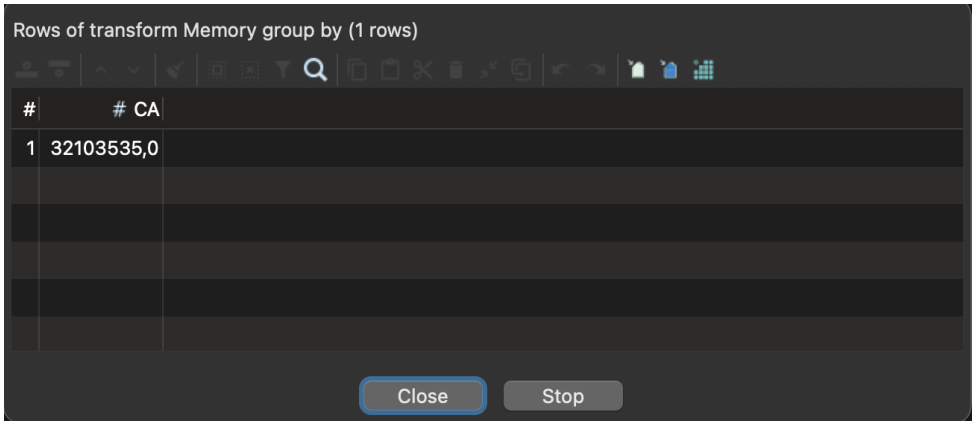


FIGURE 34 – Prévisualisation : une seule ligne contenant le chiffre d'affaires total.

6.2 Chiffre d'affaires par dimension

La formule générale est

$$\mathrm{CA}_{\text{groupe}} = \sum_{i \in \text{groupe}} \mathrm{CA}_i$$

Quatre agrégations sont construites en parallèle à partir du flux issu du *Calculator* :

Transformation	**Champ de regroupement**	**Sortie**
Memory group by client_id	`client_id`	*Text file output 2*
Memory group by produit_id	`produit_id`	*Text file output*
Memory group by ville	`ville`	*Text file output 3*
Memory group by date	`date`	*Text file output 4*

TABLE 3 – Agrégations du pipeline.

Attention

Un champ qui n'est ni un champ de regroupement ni un agrégat est perdu après un *Memory group by*. Pour conserver par exemple `nom` et `ville` avec le CA par client, il faut les ajouter aux champs de regroupement.

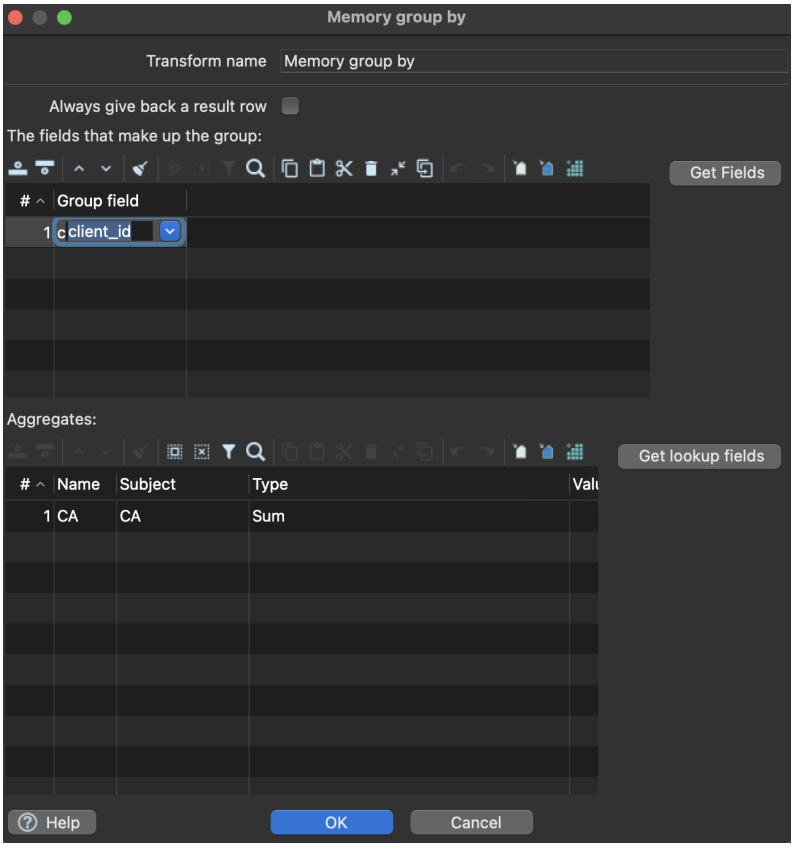


FIGURE 35 – Agrégation du CA par client (regroupement sur `client_id`).

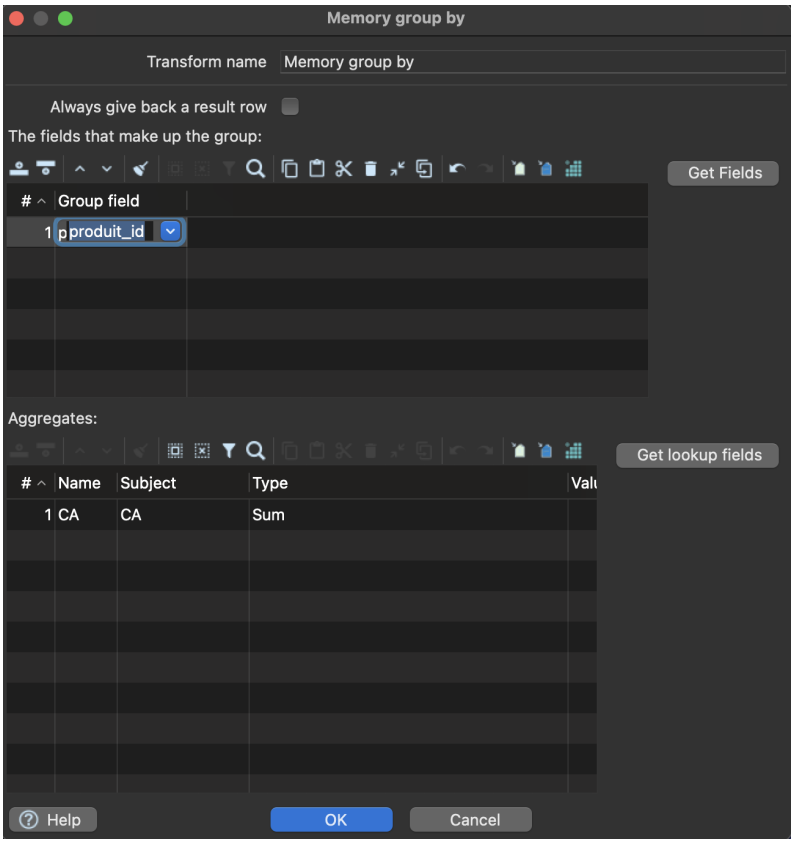


FIGURE 36 – Agrégation du CA par produit (regroupement sur `produit_id`).

6.3 La transformation *Text file output*

Text file output

Transformation de sortie qui écrit le flux de lignes dans un fichier texte (CSV). On configure le nom du fichier, l'extension, le séparateur, l'en-tête et les champs à écrire.

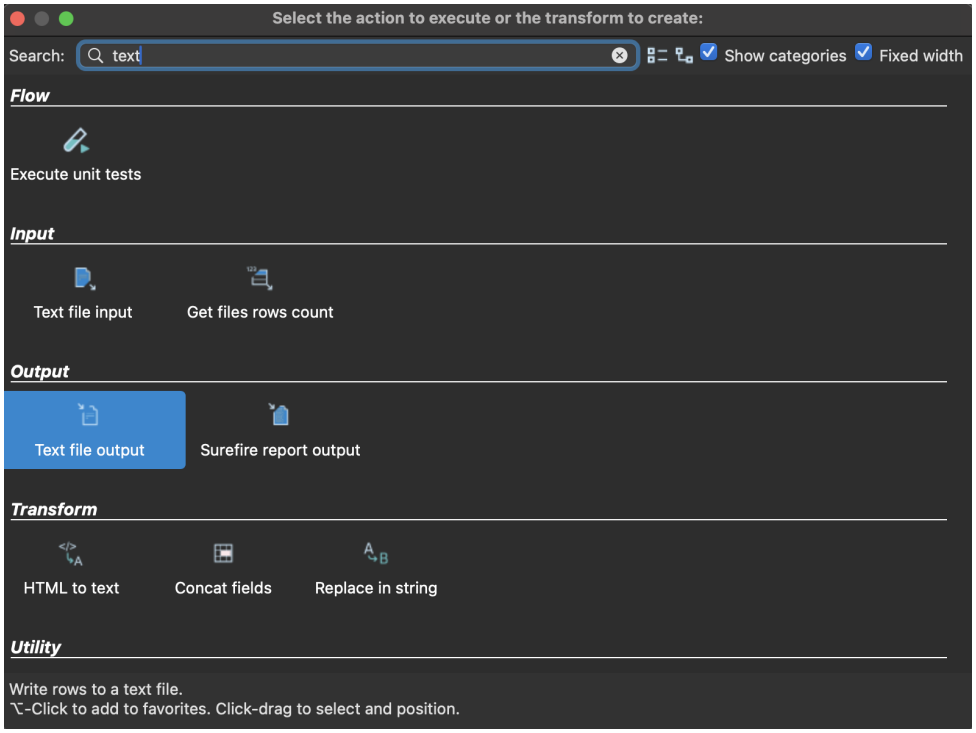


FIGURE 37 – Ajout de la transformation *Text file output*.

Les fichiers de sortie sont écrits dans le dossier `data/` : `ca_client.txt`, `ca_produit.txt`, `ca_ville.txt` et `ca_date.txt`. Dans la transformation, on renseigne par exemple le nom de fichier `${PROJECT_HOME}/data/ca_client.txt`, avec l'extension `txt` (le contenu reste un fichier délimité).

6.4 Pipeline ETL complet

Le pipeline complet, enregistré sous le nom `data/pipeline_test.hpl`, est celui de la figure 1 présentée au début de ce document. Le tableau suivant en donne la lecture, branche par branche.

Branche	Transformations	Rôle
Commandes	CSV file input commandes → Filter rows → Sort rows 2	Lire, ne garder que les commandes valides, trier sur `client_id`
Clients	CSV file input Clients → Filter rows 2 → Sort rows → Unique rows → Sort rows 4	Lire, filtrer les données incomplètes, trier, dédupliquer, retrier pour la jointure
Produits	CSV file input produits → Filter rows 3 → Sort rows 3	Lire, supprimer les produits sans prix, trier sur `produit_id`
Jointures	Merge join, Sort rows 5, Merge join 2	Enrichir les commandes avec les clients, puis avec les produits
Préparation	Select values → Calculator	Garder les bons champs, calculer `CA`
Sorties	4 × (Memory group by → Text file output)	Agréger et exporter

Table 4 – Lecture du pipeline complet.

7 Étape 5 – Orchestration avec un workflow

Le pipeline réalise le traitement des données. Pour l'automatiser et l'intégrer dans une chaîne de traitement, on utilise un **workflow**.

7.1 Création du workflow

Cliquez sur « + » puis sur « Workflow ». Le workflow contient dès le départ l'action *Start*, point d'entrée de l'exécution. Enregistrez-le sous le nom `data/workflow_test.hwf`.

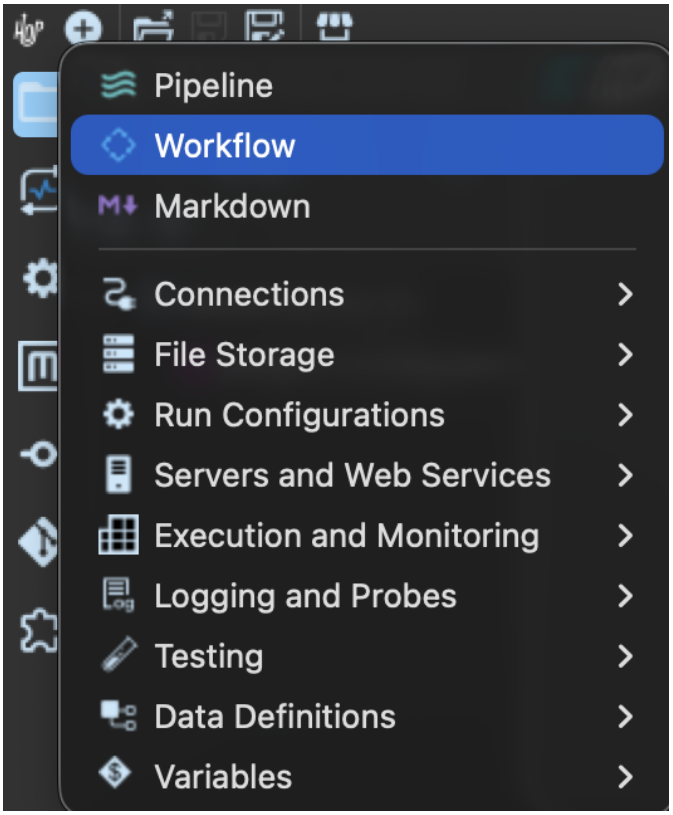


FIGURE 38 – Création d'un nouveau workflow.

7.2 L'action *Pipeline*

Action Pipeline

Action de workflow qui exécute un fichier `.hpl`. On précise le chemin du pipeline et la configuration d'exécution. L'action réussit si le pipeline se termine sans erreur et échoue sinon.

Cliquez sur le canevas, recherchez « pipe » et ajoutez l'action *Pipeline*.

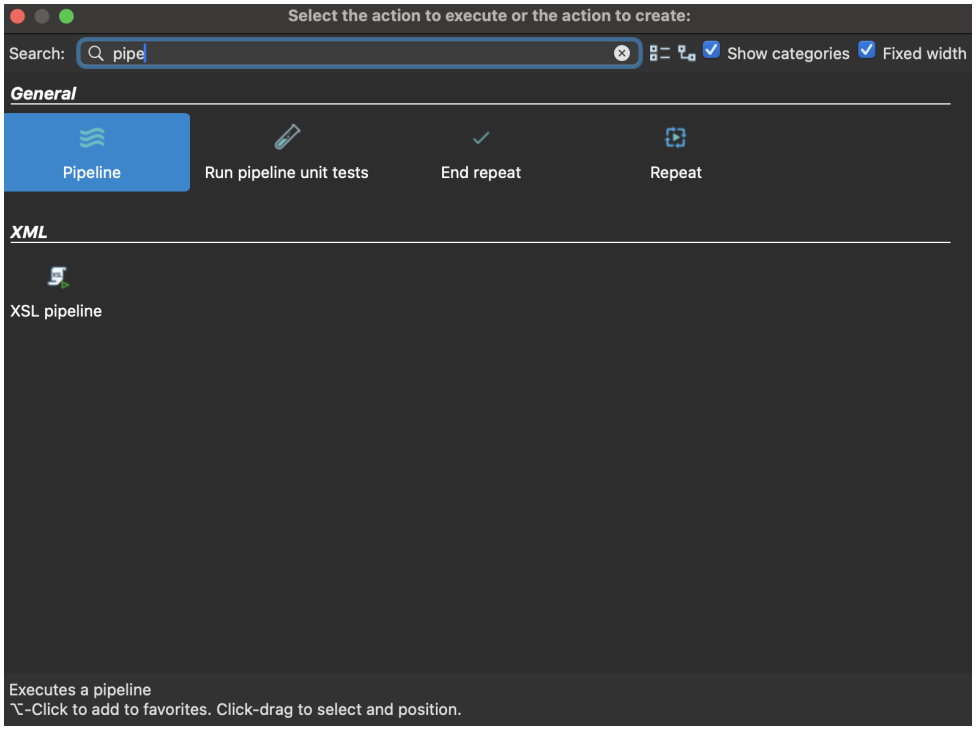


FIGURE 39 – Ajout d'une action *Pipeline* dans le workflow.

Dans la configuration de l'action, sélectionnez le fichier `pipeline_test.hpl` situé dans le dossier `data/`.

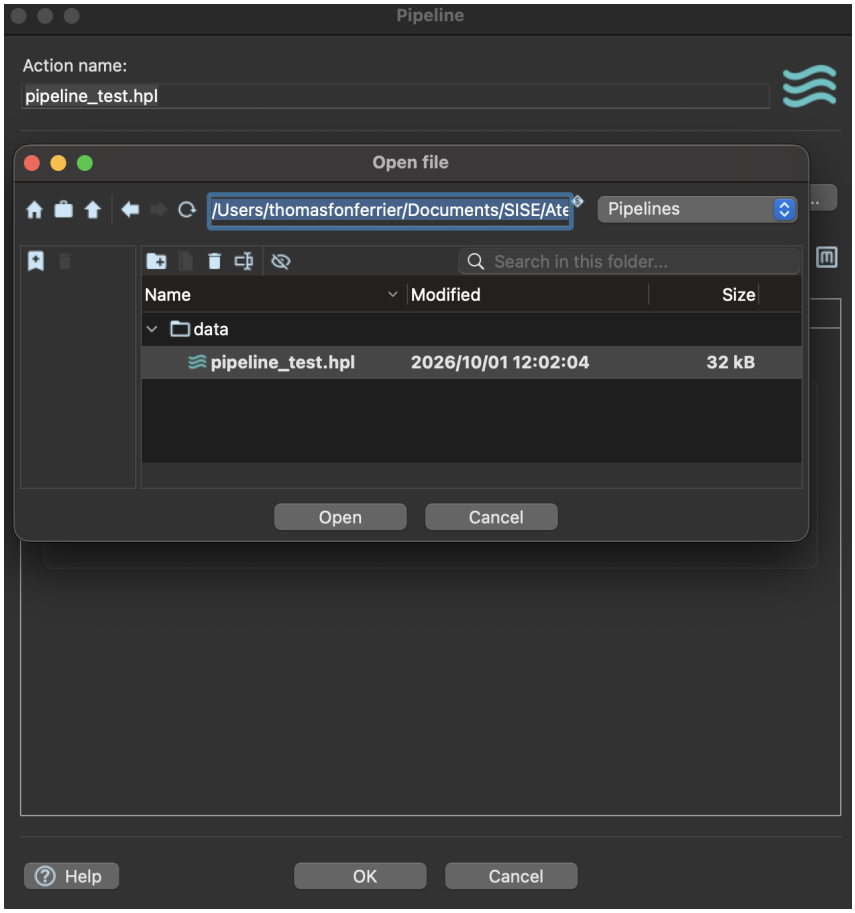


FIGURE 40 – Sélection du fichier pipeline à exécuter.

7.3 Les hops de workflow

Contrairement aux hops de pipeline (flux de données), les hops d'un workflow définissent la *condition de poursuite*. Reliez l'action *Start* à l'action *Pipeline* par un hop.

Type de hop	Signification
Inconditionnel	Passer à l'action suivante quoi qu'il arrive
Succès (vert)	Passer à l'action suivante uniquement si l'action précédente réussit
Échec (rouge)	Passer à l'action suivante uniquement si l'action précédente échoue

TABLE 5 – Types de hops dans un workflow.

Gestion des erreurs – que se passe-t-il si le pipeline de nettoyage échoue ?

Les étapes suivantes ne doivent pas être exécutées : une jointure ou une agrégation réalisée sur des données non nettoyées produirait des résultats faux sans que l'on s'en rende compte. En utilisant des hops de *succès*, la chaîne s'arrête à la première erreur. On peut en outre relier un hop d'*échec* vers une action d'alerte (message, log, mail) ou une action *Abort*.

7.4 Exécution du workflow

Le lancement du workflow (bouton en forme de triangle) exécute les actions dans l'ordre. L'onglet de logs indique le statut de chaque action et le résultat global. En cas de succès, les fichiers de résultats (`ca_client.txt`, `ca_produit.txt`, `ca_ville.txt`, `ca_date.txt`) sont générés automatiquement dans le dossier `data/`, sans lancer manuellement le pipeline.

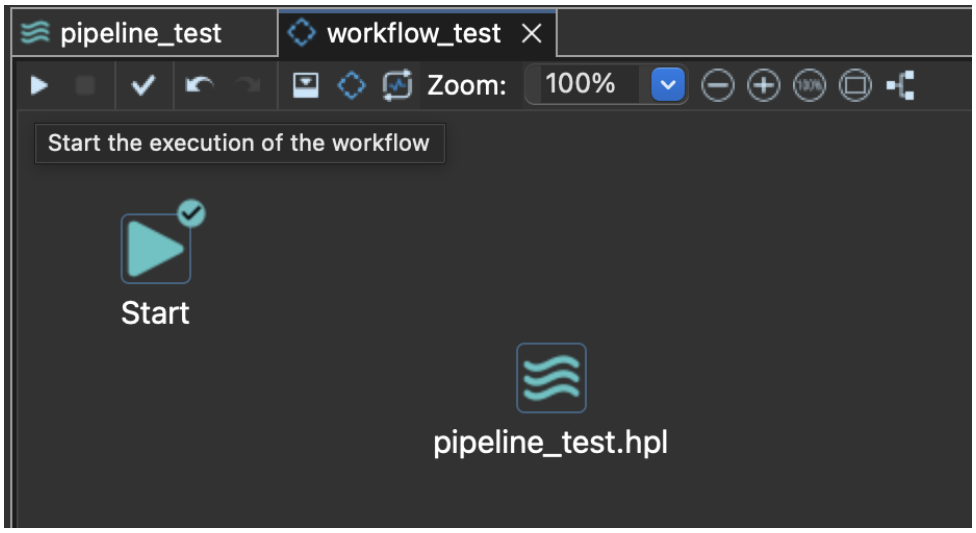


FIGURE 41 – Le workflow `workflow_test` : actions *Start* et *Pipeline*.

8 Étape 6 – En autonomie : segmentation premium

Cette dernière partie est volontairement plus ouverte : un énoncé, quelques pistes, puis une solution possible à comparer avec votre réalisation.

8.1 Énoncé

Le service marketing souhaite identifier les **clients premium**, c'est-à-dire ceux dont le chiffre d'affaires total dépasse un seuil. Le fichier `data/ca_client.txt` contient déjà le CA par client.

Mission

Créer un pipeline `data/segmentation_premium.hpl` qui lit `clients.csv` et `ca_client.txt`, associe chaque client à son CA, ajoute une colonne `premium` (`Oui` si le CA est supérieur ou égal à un seuil, `Non` sinon) et exporte le résultat dans `data/clients_premium.csv`. Puis l'ajouter au workflow, après le pipeline ETL.

Contraintes : le seuil doit être paramétrable (pas écrit en dur) ; aucun client ne doit disparaître, y compris ceux sans commande valide (leur CA vaut alors 0) ; les doublons de `clients.csv` doivent être traités avant la jointure.

Pour aller plus loin (libre). Ajoutez au workflow un pipeline de statistiques par segment, un pipeline des 5 meilleurs produits, des actions de contrôle (fichiers présents) et une branche d'échec qui interrompt proprement le traitement.

8.2 Pistes

Besoin	Transformation
Lire `ca_client.txt`	*CSV file input* (même séparateur et même format décimal que l'écriture d'origine)
Garder tous les clients	*Merge join* de type LEFT OUTER (flux triés), clients en première entrée
Remplacer un CA absent par 0	*If field value is null*
Seuil paramétrable	Paramètre `SEUIL_PREMIUM` lu par *Get variables* (champ `seuil`)
Créer la colonne `premium`	*Filter rows* (`CA` $\geq$ `seuil`), deux *Add constants*, puis *Append streams*
Exporter	*Select values*, *Sort rows*, *Text file output*

Table 6 – Transformations utiles pour la segmentation premium.

8.3 Solution proposée

```
Clients : CSV file input -> Filter rows -> Sort rows -> Unique rows
CA      : CSV file input (ca_client.txt) -> Sort rows
Suite   : Merge join (LEFT OUTER) -> If field value is null
          -> Get variables -> Filter rows (CA >= seuil)
          -> Add constants (Oui) / Add constants (Non)
          -> Append streams -> Select values -> Sort rows
          -> Text file output (clients_premium.csv)
```

Trois points méritent attention :

— la jointure est de type **LEFT OUTER** : avec une jointure INNER, les clients sans commande disparaîtraient ;

— le seuil est un **paramètre** du pipeline (valeur par défaut, par exemple 500, à ajuster selon la distribution des CA), transmis par le workflow : changer de seuil ne demande aucune modification du pipeline ;

— comme à l'étape 3, le champ `client_id` du second flux est renommé `client_id_1` par la jointure : il est retiré avec *Select values*.

8.4 Pipelines complémentaires et workflow étendu

— `stats_premium.hpl` : lit `clients_premium.csv`, puis un *Memory group by* sur `premium` calcule le nombre de clients (*Count*), le CA cumulé (*Sum*) et le CA moyen (*Average*) ; export dans `data/stats_premium.csv`.

— `top_produits.hpl` : lit `ca_produit.txt`, le trie par CA décroissant, ne garde que les lignes `1..5` avec *Sample rows* ; export dans `data/top5_produits.csv`.

Le workflow étendu enchaîne ces pipelines après le pipeline ETL, avec un contrôle préalable des fichiers sources (*Checks if files exist*) et une branche d'échec : en cas d'erreur, un message est écrit dans le journal (*Write to log*) puis le workflow est interrompu (*Abort workflow*).

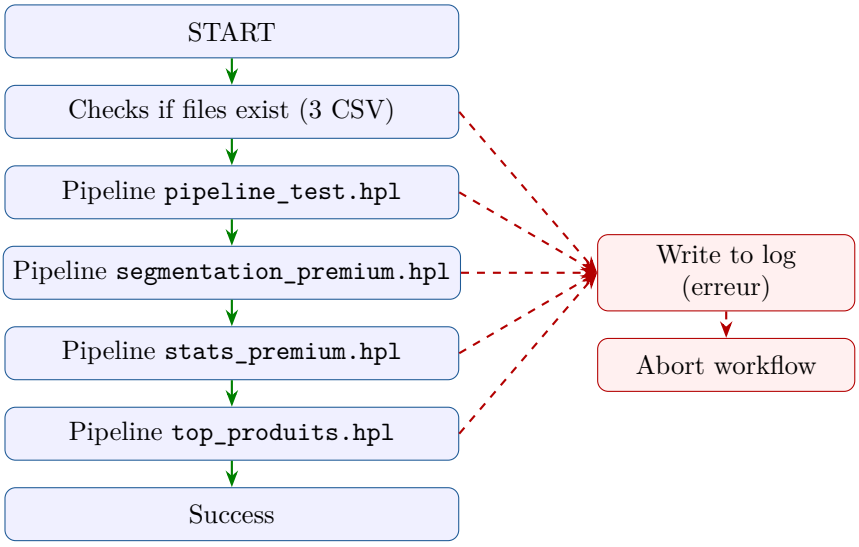


FIGURE 42 – Workflow étendu : enchaînement nominal (flèches pleines) et branche d'échec (pointillés).

Dans les propriétés du workflow, on déclare le paramètre `SEUIL_PREMIUM` ; l'onglet des paramètres de l'action *Pipeline* de la segmentation l'associe au paramètre du pipeline (valeur `${SEUIL_PREMIUM}`).

Vérifications.

- `clients_premium.csv` contient autant de lignes que de clients uniques dans `clients.csv` ;
- tous les clients « Oui » ont un CA supérieur ou égal au seuil, tous les clients « Non » un CA strictement inférieur ;
- en changeant le seuil, le nombre de clients premium varie dans le bon sens ; en supprimant `ca_client.txt`, la branche d'échec s'active.

9 Questions de synthèse

1. **Quelle est la différence entre un pipeline et un workflow ?**
   Un pipeline transforme des données (flux de lignes entre transformations, exécutées en parallèle). Un workflow orchestre l'exécution de tâches (actions séquentielles, conditionnées par leur succès ou leur échec).

2. **Pourquoi nettoyer les données avant les jointures ?**
   Pour éviter qu'un doublon multiplie les lignes, qu'une clé manquante fasse disparaître ou fausse des résultats, ou qu'une valeur invalide se propage dans les calculs.

3. **Quelle est la clé entre les commandes et les clients ?**
   `client_id`.

4. **Quelle est la clé entre les commandes et les produits ?**
   `produit_id`.

5. **Quelle transformation calcule le chiffre d'affaires ?**
   Calculator (opération $A \times B$ sur `quantite` et `prix`).

6. **Quelle transformation calcule le CA par ville ?**
   Memory group by (regroupement sur `ville`, somme de `CA`).

7. **Quelle est la différence entre une transformation et une action de workflow ?**
   Une transformation appartient à un pipeline et manipule des lignes de données. Une action appartient à un workflow et exécute une tâche (lancer un pipeline, envoyer un mail, copier un fichier…) en retournant un succès ou un échec.

8. **Quels avantages Hop présente-t-il par rapport à un script Python unique ?**
   Conception visuelle et auto-documentée, modularité et réutilisation, gestion native des erreurs dans le workflow, logs et monitoring intégrés, exécution parallèle des transformations, paramétrage par variables et environnements, maintenance facilitée pour des non-développeurs. Le script Python reste plus flexible pour des logiques très spécifiques.

10 Bilan

Ce tutoriel a permis de parcourir l'ensemble de la chaîne :

— créer un projet et un pipeline Apache Hop ;

— importer et inspecter des fichiers CSV, avec l'aide du bouton *Get fields* ;

— filtrer, trier et dédupliquer les données ;

— joindre des flux avec *Merge join* (en respectant le tri) ;

— calculer une colonne avec *Calculator* ;

— agréger avec *Memory group by* et exporter avec *Text file output* ;

— orchestrer l'ensemble avec un workflow et gérer les échecs ;

— réutiliser le résultat d'un pipeline dans un autre, paramétrer un traitement et étendre un workflow (partie en autonomie).

Pour aller plus loin : le découpage du traitement en plusieurs pipelines (nettoyage, jointure, agrégations) orchestrés par le workflow, et la gestion des lignes rejetées dans un flux d'erreurs sont les extensions naturelles de ce travail.